

Portal Web de Gestión de Incidencias

Con Autenticación LDAP

Nombre del alumno o de la alumna: Jorge Salmerón Carrasco

Curso académico: 2º de ASIR

Tutora/Tutor del proyecto: Alejandro de Abreu

ÍNDICE PAGINADO

1. RESUMEN	3
2. JUSTIFICACIÓN DEL PROYECTO	4
3. OBJETIVOS	4
a. OBJETIVO GENERAL	5
b. OBJETIVOS ESPECÍFICOS	5
4. DESARROLLO	7
a. FUNDAMENTACIÓN TEÓRICA	7
b. MATERIALES Y MÉTODOS	10
c. RESULTADOS Y ANÁLISIS	13
5. CONCLUSIONES	22
6. LÍNEAS DE INVESTIGACIÓN FUTURAS	26
7. BIBLIOGRAFÍA	27
8. ANEXOS	28
9. OTROS PUNTOS	51
a. Aportaciones personales	51
b. Retos profesionales	51
c. Retos personales	51
d. Agradecimientos	52

1. RESUMEN

Hoy en día, cualquier empresa que quiera gestionar de forma eficiente sus incidencias informáticas necesita algo más que un correo electrónico o una llamada de teléfono. Necesita un sistema centralizado, trazable y seguro. Este proyecto nace de esa necesidad real.

He desarrollado un portal web de gestión de incidencias basado en GLPI, conectado a un directorio OpenLDAP para que todos los usuarios del sistema se autentiquen con un único usuario y contraseña corporativa. La infraestructura se despliega sobre dos servidores virtualizados en VirtualBox con Ubuntu Server 24.04, simulando un entorno empresarial real.

El proyecto va más allá de una simple instalación. Se ha automatizado la creación de tickets mediante un script de Python que usa la API REST de GLPI, se ha desarrollado un portal web con Flask para simplificar la experiencia del usuario final, y se han aplicado medidas de seguridad como un WAF con reglas OWASP, auditorías del sistema con Lynis y Nikto, y protección contra ataques de fuerza bruta con fail2ban. A propuesta del tutor, se implementó un sistema de copias de seguridad automáticas nocturnas con subida a Dropbox y un sistema de asignación automática de tickets según el departamento. Adicionalmente se añadieron tres mejoras propias: clasificación automática de la urgencia de los tickets por palabras clave, un bot de Telegram que avisa al técnico en tiempo real cuando llega un ticket nuevo, y un servidor DNS con BIND9 que permite acceder a los servicios por nombre de dominio con HTTPS.

El resultado es un sistema funcional que demuestra competencias en administración de sistemas Linux, seguridad informática, servicios en red y desarrollo web, las cuatro áreas fundamentales del ciclo ASIR.

2. JUSTIFICACIÓN DEL PROYECTO

La gestión de incidencias informáticas es una necesidad real en cualquier organización. Sin un sistema adecuado, las peticiones de los usuarios se pierden, no hay trazabilidad de lo que está pendiente y es imposible medir el rendimiento del equipo técnico. Esta situación, que viven a diario muchas empresas medianas, justifica por sí sola la implantación de una herramienta como GLPI.

Sin embargo, instalar un sistema de ticketing no es suficiente si no va acompañado de una gestión centralizada de identidades. Que cada empleado tenga contraseñas distintas para cada aplicación es un problema de seguridad y administración. La integración con OpenLDAP resuelve esto: un único par usuario/contraseña da acceso a todos los servicios corporativos.

Elegí este proyecto por tres razones concretas. Primera, porque las tecnologías que utiliza son las que se demandan en el mercado laboral: GLPI, OpenLDAP, Apache y Linux están presentes en la gran mayoría de departamentos de sistemas. Segunda, porque abarca de forma práctica y real todas las asignaturas del ciclo ASIR. Tercera, porque me permitía aplicar conocimientos de seguridad informática en un entorno con sentido profesional real, no en ejercicios aislados.

Las dos mejoras propuestas por el tutor refuerzan el enfoque profesional del proyecto. El sistema de backup automático con subida a Dropbox garantiza que los datos estén protegidos incluso si el servidor falla, ya que siempre hay una copia reciente en la nube. La asignación automática de tickets por departamento elimina el trabajo manual de enrutamiento, que es una de las tareas más repetitivas en cualquier help desk real.

3. OBJETIVOS

a. OBJETIVO GENERAL

Diseñar, desplegar y configurar un sistema completo de gestión de incidencias basado en GLPI con autenticación centralizada mediante OpenLDAP, automatización de procesos, seguridad avanzada, las mejoras funcionales propuestas por el tutor y mejoras adicionales propias que elevan el nivel profesional del sistema.

b. OBJETIVOS ESPECÍFICOS

1. Configurar dos servidores virtualizados con Ubuntu Server 24.04 en VirtualBox con redes correctamente segmentadas.
2. Instalar y configurar OpenLDAP con estructura de unidades organizativas, grupos y usuarios.
3. Instalar y configurar GLPI integrado con Apache2, MariaDB y PHP.
4. Integrar GLPI con el directorio LDAP para autenticación centralizada de usuarios.
5. Activar la API REST de GLPI y generar los tokens de acceso necesarios.
6. Desarrollar un script Python que cree tickets automáticamente en GLPI a través de la API REST.
7. Aplicar medidas de seguridad: WAF con ModSecurity y reglas OWASP, cabeceras HTTP, auditorías con Lynis y Nikto, y protección con fail2ban.
8. Desarrollar un portal web con Flask que permita a los usuarios abrir incidencias autenticándose con sus credenciales LDAP.
9. Implementar un sistema de backup automático nocturno de la base de datos con retención de 7 días y subida automática a Dropbox.
10. Configurar reglas de asignación automática de tickets según la categoría del departamento.
11. Implementar clasificación automática de la urgencia de los tickets mediante análisis de palabras clave.

12. Desarrollar un bot de Telegram que notifique al técnico en tiempo real cuando se crea un ticket nuevo.
13. Configurar un servidor DNS con BIND9 y HTTPS para acceder a los servicios por nombre de dominio seguro.

4. DESARROLLO

a. FUNDAMENTACIÓN TEÓRICA

En este apartado se describen los procedimientos, tecnologías y decisiones técnicas que fundamentan el proyecto, explicando qué se va a hacer, cómo y por qué.

GLPI — Gestión de Incidencias y Activos

GLPI es la plataforma open source de gestión de servicios TI sobre la que se construye todo el sistema. Escrita en PHP, ofrece gestión de tickets, inventario de activos y una API REST completa. Se eligió por ser software libre, ampliamente usado en entornos empresariales reales y por su capacidad de integrarse con directorios LDAP. La versión utilizada es la 10.0.16 sobre una pila Apache2, MariaDB y PHP 8.3.

OpenLDAP y el protocolo LDAP

LDAP es el protocolo estándar para gestionar directorios de usuarios en red. Organiza la información en una estructura jerárquica en árbol donde cada entrada tiene un identificador único llamado DN. OpenLDAP es la implementación más extendida en entornos Linux y la elegida para este proyecto. La estructura del directorio tiene como raíz `dc=empresa,dc=local`, con las ramas `ou=usuarios` y `ou=grupos`, y los usuarios `admin_glpi`, `tecnico1` y `user_tfg`.

API REST de GLPI

La API REST de GLPI permite que aplicaciones externas interactúen con el sistema mediante peticiones HTTP estándar. Es el componente clave que hace posible el Zero-Touch Ticketing: el script Python la usa para crear tickets sin ninguna intervención humana. La autenticación requiere un App Token que identifica a la aplicación y un User Token que identifica al usuario.

ModSecurity y OWASP CRS

ModSecurity es el módulo de Apache que actúa como Web Application Firewall, analizando cada petición HTTP antes de que llegue a GLPI y bloqueando las que coinciden con patrones de ataque. Las reglas OWASP CRS son el conjunto estándar que protege contra los ataques más comunes del OWASP Top 10: inyección SQL, XSS e inclusión de ficheros remotos.

Flask

Flask es un microframework web para Python que se usó para desarrollar el portal personalizado de creación de incidencias. Actúa como capa intermedia entre el usuario y GLPI: valida las credenciales contra el directorio LDAP y llama a la API de GLPI para crear el ticket. El usuario nunca necesita entrar en la interfaz de GLPI.

Backup automático con mysqldump y cron

mysqldump es la herramienta estándar de MariaDB para realizar volcados completos de bases de datos. Combinado con gzip para comprimir el resultado y con cron para programar la ejecución, permite implementar un sistema de copias de seguridad nocturnas completamente automático. El script desarrollado guarda los backups durante 7 días y elimina los más antiguos de forma automática. Adicionalmente, rclone se encarga de subir cada copia a Dropbox nada más crearla, garantizando que siempre existe una copia en la nube independiente del servidor local.

Reglas de negocio en GLPI

GLPI dispone de un motor de reglas que ejecuta acciones automáticas cuando se crea un ticket. Se configuraron cuatro reglas que detectan la categoría del ticket (Hardware, Software, Redes o Sistemas) y asignan automáticamente el grupo técnico Soporte, eliminando el trabajo manual de enrutamiento.

Clasificación automática de urgencia por palabras clave

Para mejorar la experiencia del usuario y reducir los errores en la asignación de prioridades, se implementó un sistema de clasificación automática de la urgencia de los tickets. El sistema analiza el título y la descripción que escribe el usuario y busca palabras clave predefinidas que indican distintos niveles de gravedad. Si el texto contiene palabras como "caído", "urgente" o "no puedo trabajar" asigna urgencia muy alta. Si contiene palabras como "error", "fallo" o "lento" asigna urgencia alta. Si contiene palabras como "problema" o "necesito ayuda" asigna urgencia media. En cualquier otro caso asigna urgencia baja por defecto. Esto elimina el selector manual de urgencia del formulario y garantiza una clasificación más objetiva y consistente.

Bot de Telegram

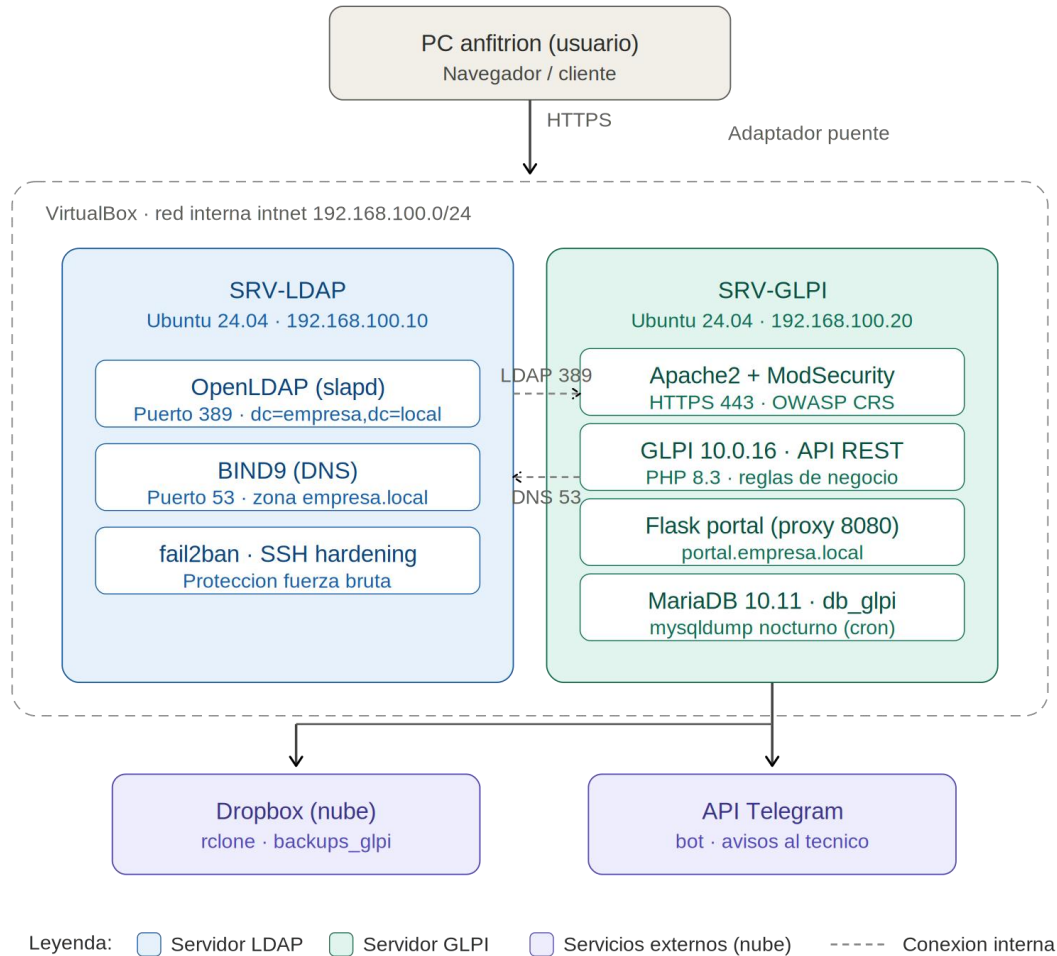
Telegram dispone de una API pública que permite crear bots capaces de enviar mensajes automáticamente a cualquier usuario o grupo. En este proyecto se creó un bot mediante BotFather, la herramienta oficial de Telegram para gestionar bots, que proporciona un token de autenticación único. Cada vez que se crea un ticket desde el portal Flask, el sistema llama a la API de Telegram y manda un mensaje al técnico con el número de ticket, el usuario que lo abrió, el título, la categoría y la urgencia asignada automáticamente. Esto permite al técnico estar informado en tiempo real sin necesidad de estar revisando GLPI constantemente.

Servidor DNS con BIND9

BIND9 es el servidor DNS más utilizado en entornos Linux y es el estándar de facto en la administración de sistemas. Un servidor DNS traduce nombres de dominio como glpi.empresa.local a direcciones IP, de la misma forma que los servidores DNS de internet traducen google.com a una IP. En este proyecto se instaló BIND9 en el servidor LDAP y se configuró una zona de resolución directa para el dominio empresa.local, de forma que glpi.empresa.local apunta al servidor GLPI y portal.empresa.local apunta al portal Flask. Esto elimina la necesidad de recordar las IPs de los servidores y simula fielmente un entorno empresarial real.

HTTPS con certificado SSL autofirmado

HTTPS es la versión segura del protocolo HTTP que cifra todas las comunicaciones entre el navegador y el servidor mediante TLS. Para implementarlo se generó un certificado SSL autofirmado con OpenSSL y se configuró Apache para servir GLPI sobre HTTPS en el puerto 443, redirigiendo automáticamente cualquier acceso por HTTP al puerto seguro. Para el portal Flask se configuró Apache como proxy inverso, de forma que las peticiones a https://portal.empresa.local llegan a Apache que las redirige internamente a Flask. En un entorno de producción real se usaría un certificado emitido por una CA reconocida, pero para este entorno de laboratorio el certificado autofirmado es suficiente y demuestra el proceso completo de configuración.



b. MATERIALES Y MÉTODOS: estrategias de búsqueda, metodología y técnicas utilizadas.

Infraestructura de hardware y virtualización

Todo el proyecto se despliega sobre máquinas virtuales en Oracle VirtualBox. Esta decisión permite replicar un entorno empresarial real sin hardware dedicado y facilita la recuperación ante errores mediante snapshots. La configuración de red usa dos adaptadores en cada servidor: uno en red interna (intnet, 192.168.100.0/24) para la comunicación entre servidores, con el servidor LDAP en 192.168.100.10 y el servidor GLPI en 192.168.100.20, y otro en adaptador puente para el acceso desde el exterior y la descarga de paquetes.

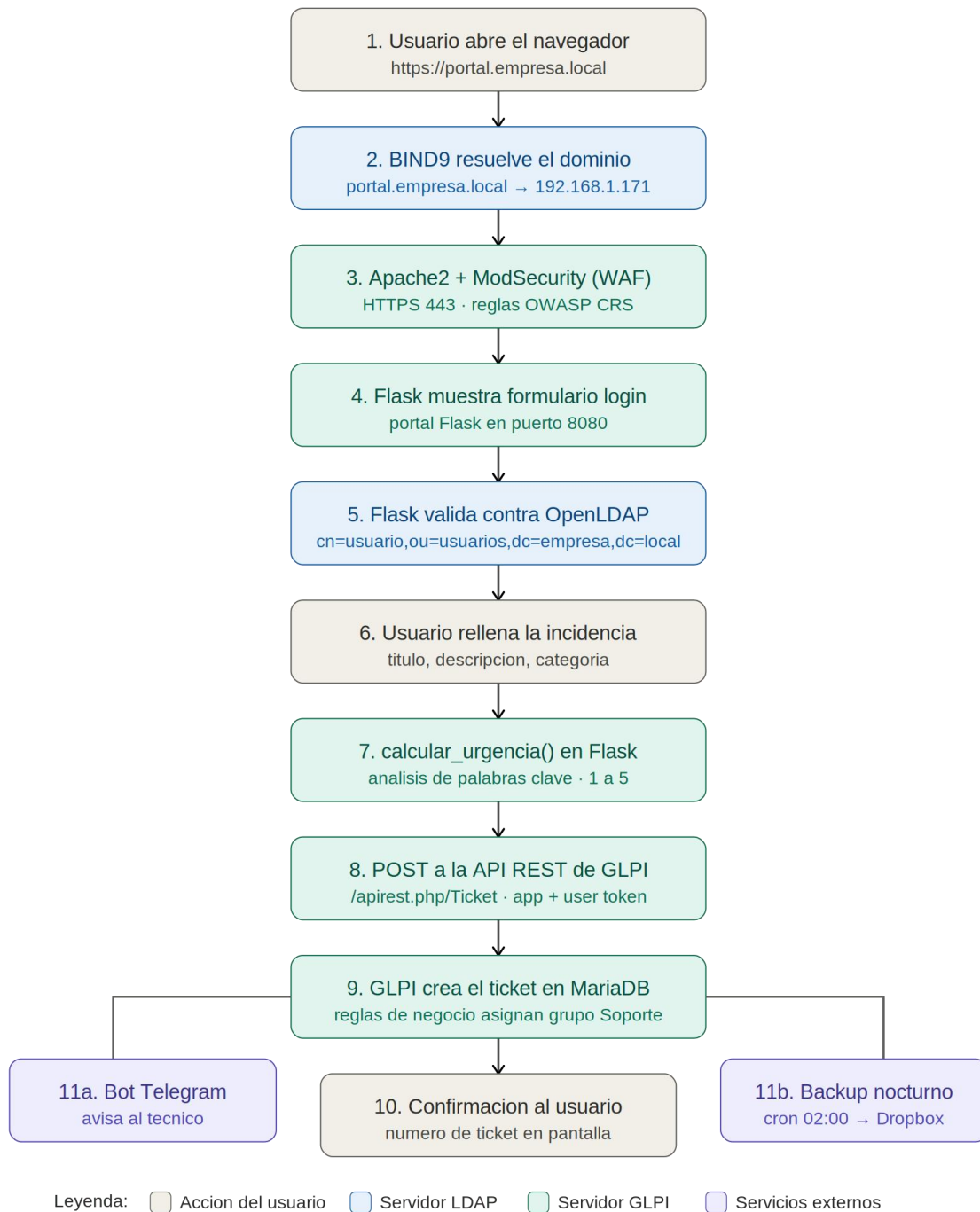
Software utilizado

Software	Versión	Función
Oracle VirtualBox	7.2	Hipervisor de virtualización
Ubuntu Server	24.04 LTS	Sistema operativo de ambos servidores
OpenLDAP (slapd)	2.6	Servicio de directorio LDAP
Apache2	2.4.58	Servidor web
MariaDB	10.11	Base de datos relacional
PHP	8.3	Lenguaje de scripting web
GLPI	10.0.16	Plataforma de gestión de incidencias
Python 3	3.12	Lenguaje para el script de automatización
Flask	3.1.x	Framework web para el portal personalizado
ModSecurity	3.0.9	Web Application Firewall
OWASP CRS	4.19	Reglas de seguridad para ModSecurity
Lynis	3.1	Herramienta de auditoría del sistema
Nikto	2.5.0	Escáner de vulnerabilidades web
Rclone	1.74.1	Subida automática de backups a Dropbox.
BIND9	9.18	Servidor DNS para resolución de dominios.
OpenSSL	3.0.13	Generación de certificados SSL.

Metodología de trabajo

Se siguió una metodología incremental por fases verificables de forma independiente: preparación de infraestructura, instalación de dependencias, instalación de GLPI, integración LDAP, API REST, automatización, seguridad, portal Flask, backup automático y reglas de asignación.

c. RESULTADOS Y ANÁLISIS



Servidor LDAP

```
jorge@srv-ldap: ~
jorge@srv-ldap:~$ ldapsearch -x -LLL -H ldap://localhost -D "cn=admin,dc=empresa,dc=local" -W -b "dc=emp
resa,dc=local" "(objectClass=inetOrgPerson)" cn
Enter LDAP Password:
dn: cn=admin_glpi,ou=usuarios,dc=empresa,dc=local
cn: admin_glpi

dn: cn=tecnico1,ou=usuarios,dc=empresa,dc=local
cn: tecnico1

dn: cn=user_tfg,ou=usuarios,dc=empresa,dc=local
cn: user_tfg

jorge@srv-ldap:~$
```

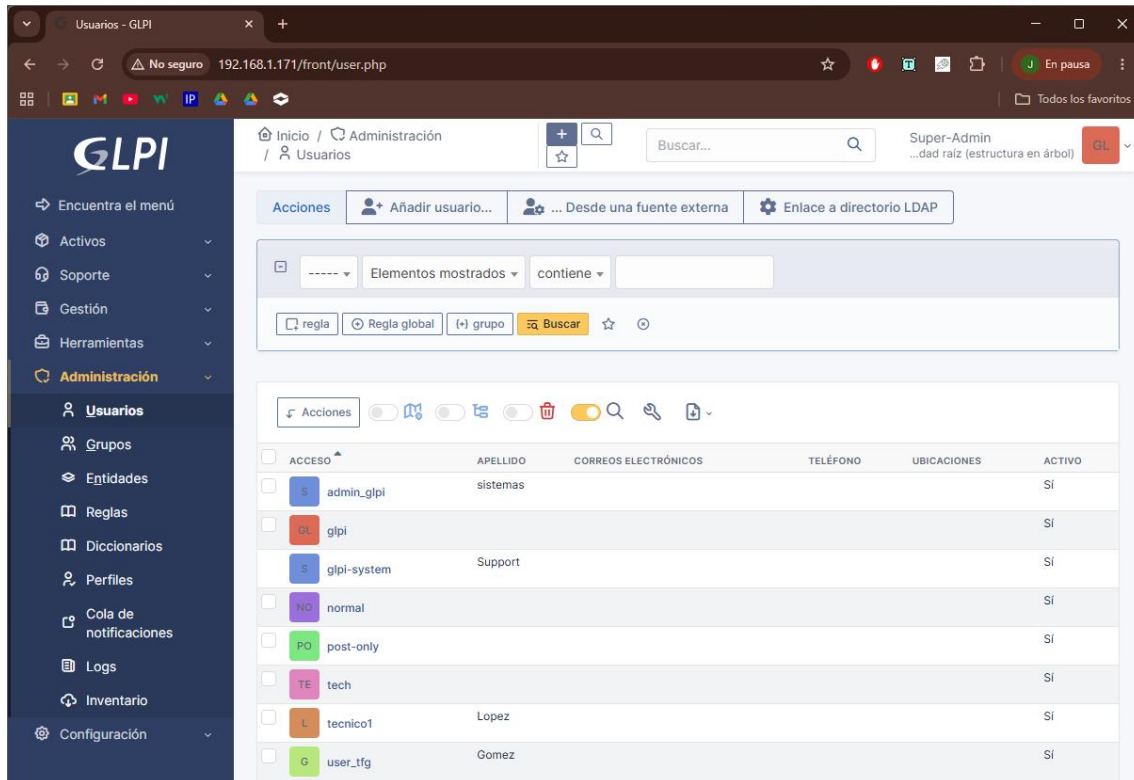
El primer resultado visible del proyecto es el directorio OpenLDAP funcionando con la estructura de usuarios que necesitamos. El directorio tiene el dominio empresa.local y contiene los tres usuarios del sistema: admin_glpi, tecnico1 y user_tfg, cada uno con su contraseña almacenada de forma segura en formato hash, nunca en texto plano. También contiene los grupos soporte y clientes. Todo esto se verificó con el comando ldapsearch, que consultó el directorio y devolvió correctamente todos los objetos creados.

El servidor GLPI y la integración con LDAP

GLPI quedó instalado y accesible desde el navegador. Tras la instalación se eliminó por seguridad la carpeta de instalación del servidor, que si se dejara podría permitir a alguien reinstalar el sistema y borrar todos los datos. También se cambiaron todas las contraseñas por defecto que crea GLPI, que son públicamente conocidas y supondrían un riesgo de seguridad evidente.



La integración con el directorio LDAP se configuró indicándole a GLPI la dirección del servidor, el dominio del directorio y las credenciales necesarias para consultarlo. Los tres usuarios se importaron correctamente y se les asignaron sus roles: admin_glpi como administrador, tecnico1 como técnico de soporte y user_tfg como usuario final con acceso limitado solo a abrir sus propias incidencias. Se verificó que el inicio de sesión con las credenciales del directorio LDAP funcionaba correctamente.



API REST y automatización

```
jorge-glpi@srv-glpi: ~
jorge-glpi@srv-glpi:~$ sudo nano /opt/glpi_ticket.py
jorge-glpi@srv-glpi:~$ sudo chmod +x /opt/glpi_ticket.py
jorge-glpi@srv-glpi:~$ python3 /opt/glpi_ticket.py "Alerta: Apache caido" "El servidor web no responde"
5
[OK] Ticket #1 creado: Alerta: Apache caido
jorge-glpi@srv-glpi:~$ python3 /opt/glpi_ticket.py "Disco al 90%" "La particion /var esta al 90% de capa
cidad" 4
[OK] Ticket #2 creado: Disco al 90%
jorge-glpi@srv-glpi:~$ python3 /opt/glpi_ticket.py "Backup fallido" "El backup nocturno no se ha complet
ado" 3
[OK] Ticket #3 creado: Backup fallido
jorge-glpi@srv-glpi:~$
```

La API REST de GLPI se activó y se generaron las claves de acceso necesarias. Para verificar que funcionaba correctamente se hizo una prueba desde la terminal que devolvió un token de sesión válido, confirmando que la API responde correctamente a las peticiones externas.

El script Python de automatización se probó creando tres tickets de prueba con diferentes niveles de urgencia.

Acciones									
ID	TÍTULO	ESTADO	ÚLTIMA ACTUALIZACIÓN	FECHA DE APERTURA	PRIORIDAD	SOLICITANTE - SOLICITANTE	ASIGNADA A - TÉCNICO	CATEGORÍA	TIEMPO PARA RESOLVER
3	Backup fallido	Nuevo	11-05-2026 21:32	11-05-2026 21:32	Media				
2	Disco al 90%	Nuevo	11-05-2026 21:32	11-05-2026 21:32	Alta				
1	Alerta: Apache caído	Nuevo	11-05-2026 21:32	11-05-2026 21:32	Alta				

20 filas / página

Mostrando 1 a 3 de 3 filas

Los tres tickets aparecieron inmediatamente en GLPI con los datos correctos y el estado Nuevo, demostrando que el sistema de Zero-Touch Ticketing funciona de principio a fin.

A continuación, se muestra el script completo de automatización con la explicación de cada bloque:

```
jorge-glpi@srv-glpi: ~
# Aquí guardamos la dirección de GLPI y las dos claves que necesitamos
# para que la API nos deje entrar
GLPI_URL = "http://192.168.1.169/apirest.php"
APP_TOKEN = "8QvMLgG9ilq7UB2BhU7Iodl7u6fhfIMd3taADzel"
USER_TOKEN = "GWgonI0giWhAO2FA4GGLwpTbCwkkxtf18zyAKTYa"

def iniciar_sesion():
    # Le decimos a GLPI que queremos empezar a trabajar
    # y nos devuelve un ticket de sesión temporal
    r = requests.get(
        f"{GLPI_URL}/initSession",
        headers={
            "Authorization": f"user_token {USER_TOKEN}",
            "App-Token": APP_TOKEN
        }
    )
    r.raise_for_status()
    return r.json()["session_token"]

def cerrar_sesion(st):
    # Cuando terminamos, cerramos la sesión para no dejarla abierta
    requests.get(
        f"{GLPI_URL}/killSession",
        headers={"Session-Token": st, "App-Token": APP_TOKEN}
    )

def crear_ticket(titulo, descripcion, urgencia=3):
    # Valores posibles de urgencia:
    # 1=Muy baja 2=Baja 3=Media 4=Alta 5=Muy alta
    st = iniciar_sesion()
    try:
        # Mandamos el ticket a GLPI con todos sus datos
        r = requests.post(
            f"{GLPI_URL}/Ticket",
            headers={
                "Session-Token": st,
                "App-Token": APP_TOKEN,
                "Content-Type": "application/json"
            },
            json={
                "input": {
                    "name": titulo, # Título del ticket
                    "content": descripcion, # Descripción el ticket
                    "urgency": int(urgencia), # Nivel de urgencia
                    "type": 1 # 1 = Número de incidencia
                }
            }
        )
        # Mostramos el número de ticket que nos ha asignado GLPI
        tid = r.json().get("id", "ERROR")
        print(f"[OK] Ticket #{tid} creado: {titulo}")
    finally:
        # Cerramos la sesión pase lo que pase, aunque haya fallado algo
        cerrar_sesion(st)

# Desde aquí arranca el script cuando lo ejecutamos desde la terminal
# Podemos pasarle el título, la descripción y la urgencia como parámetros
# Si no ponemos nada usa los valores por defecto
if __name__ == "__main__":
    titulo = sys.argv[1] if len(sys.argv) > 1 else "Alerta automatica"
    desc = sys.argv[2] if len(sys.argv) > 2 else "Incidencia detectada automaticamente"
    urg = int(sys.argv[3]) if len(sys.argv) > 3 else 3
    crear_ticket(titulo, desc, urg)

jorge-glpi@srv-glpi:~$
```


Seguridad y hardening

```
jorge-glpi@srv-glpi: ~
jorge-glpi@srv-glpi:~$ curl -I "http://192.168.1.171/?id=1'OR'1'='1"
HTTP/1.1 403 Forbidden
Date: Thu, 21 May 2026 18:11:51 GMT
Server: Apache/2.4.58 (Ubuntu)
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
Content-Type: text/html; charset=iso-8859-1
```

Se realizó una prueba de ataque simulado contra el servidor para verificar que ModSecurity funcionaba correctamente. Se lanzó una petición con una inyección SQL, que es uno de los ataques más comunes contra aplicaciones web, y el servidor respondió con un error 403 bloqueando el ataque antes de que llegara a GLPI.

```
jorge-glpi@srv-glpi: /opt/portal_tfg

=====
Lynis security scan details:

Hardening index : 63 [#####          ]
Tests performed : 269
Plugins enabled : 1

Components:
- Firewall          [V]
- Malware scanner   [V]

Scan mode:
Normal [V] Forensics [ ] Integration [ ] Pentest [ ]

Lynis modules:
- Compliance status [?]
- Security audit    [V]
- Vulnerability scan [V]

Files:
- Test and debug information : /var/log/lynis.log
- Report data                : /var/log/lynis-report.dat
```

La auditoría con Lynis evaluó el sistema operativo completo y asignó una puntuación de seguridad de 63 sobre 100 tras aplicar las correcciones recomendadas. Entre las mejoras aplicadas destacan la instalación de fail2ban para bloquear ataques de fuerza bruta, el endurecimiento de la configuración SSH reduciendo el número máximo de intentos de contraseña a 3 y desactivando funciones innecesarias, y la activación del servicio de auditoría del sistema para registrar eventos de seguridad. El escáner Nikto confirmó que las cabeceras de seguridad HTTP estaban correctamente configuradas. Respecto al MFA, la versión de GLPI instalada no lo soporta de forma nativa, lo que se documenta como una limitación conocida del sistema que podría resolverse en una versión futura.

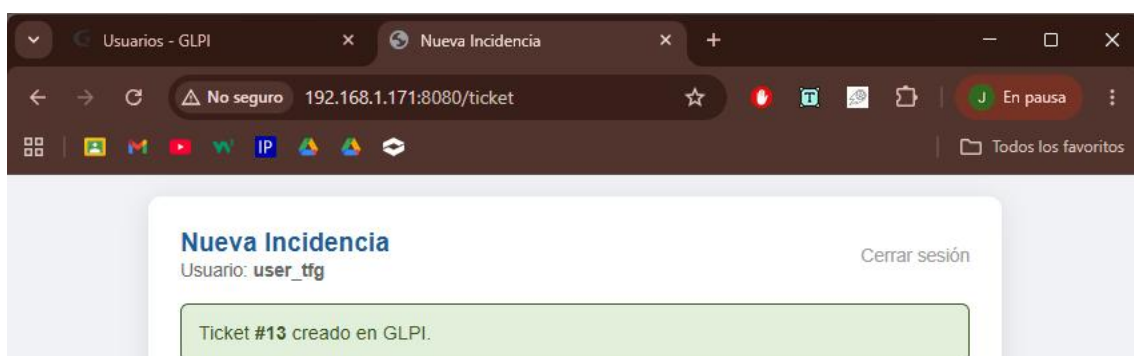
Portal Flask

El portal web personalizado quedó operativo y accesible desde el navegador. El empleado introduce su usuario y contraseña del directorio LDAP y el portal los verifica contra OpenLDAP.

Si las credenciales son correctas aparece el formulario de creación de incidencias. El empleado rellena el título, la descripción, la urgencia y la categoría del problema y pulsa enviar. En ese momento el portal crea el ticket en GLPI automáticamente y muestra al empleado el número de ticket asignado para que pueda hacer seguimiento.

			ÚLTIMA	FECHA DE		SOLICITANTE	ASIGNADA	
	ID	TÍTULO	ESTADO	ACTUALIZACIÓN	APERTURA	PRIORIDAD	SOLICITANTE	TÉCNICO
	12	Falta de puertos USB	En curso (asignada)	20-05-2026 20:21	20-05-2026 20:21	Media		
								Hardware

El empleado en ningún momento ve la interfaz de GLPI. Solo ve el portal sencillo y recibe la confirmación de que su incidencia ha quedado registrada. El ticket aparece en GLPI con la categoría correcta y el grupo Soporte asignado de forma automática, demostrando el flujo completo end-to-end funcionando.



Tras enviar el formulario, el portal muestra al usuario un mensaje de confirmación con el número de ticket asignado, cerrando así el flujo completo desde el punto de vista del usuario final.

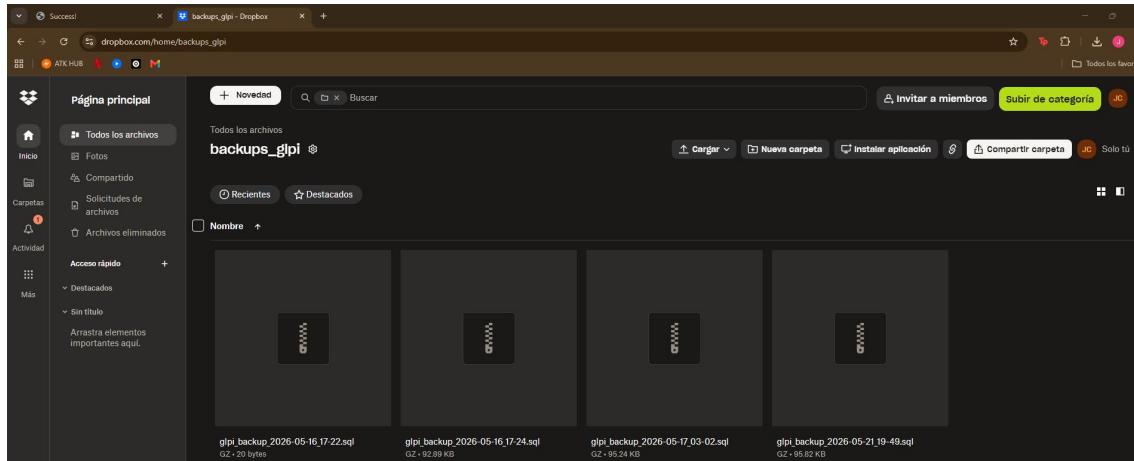
Backup automático y subida a la nube:

Para este apartado implementé dos cosas: guardar las copias de seguridad en el propio servidor y subirlas automáticamente a Dropbox para tenerlas también en la nube.

El script que creé en `/opt/backup_glpi.sh` se ejecuta cada noche a las 2:00 gracias a cron. Lo que hace es volcar toda la base de datos de GLPI, comprimirla con gzip y guardarla en `/opt/backups_glpi/` con la fecha y hora en el nombre del archivo para identificarla fácilmente. Además, borra automáticamente las copias con más de 7 días de antigüedad para que el disco no se llene con el tiempo.

```
jorge-glpi@srv-glpi:~$ sudo /opt/backup_glpi.sh
[2026-05-21_19-49] Backup creado: /opt/backups_glpi/glpi_backup_2026-05-21_19-49.sql.gz
[2026-05-21_19-49] Limpieza completada
[2026-05-21_19-49] Backup subido a Dropbox correctamente
```

Una vez guardada la copia local, rclone se encarga de subirla automáticamente a una carpeta llamada backups_glpi en mi cuenta de Dropbox. Así, aunque el servidor fallara o se perdieran los datos, siempre habría una copia reciente en la nube desde la que recuperar todo.



Para conectar rclone con Dropbox tuve que autorizar el acceso desde mi ordenador. El proceso consiste en ejecutar `rclone authorize "dropbox"` en un PC con navegador, lo que abre Dropbox automáticamente para que confirmes que das permiso a rclone. Una vez confirmado, genera un token que copié al servidor y quedó guardado en la configuración de rclone.

Verifiqué que todo funcionaba ejecutando el script manualmente y comprobando que el archivo aparecía tanto en la carpeta local del servidor como en la carpeta de Dropbox en el navegador.

Asignación automática de ticket:

☐ Asignar tickets de Hardware
 Asigna tickets de Hardware al grupo Soporte
 Añadir
 Categoría ► es ► Hardware (Entidad raíz)
 Grupo técnico ► Asignar ► Soporte
 ●
 ⋮

☐ Asignar tickets de Software
 Añadir
 Categoría ► es ► Software (Entidad raíz)
 Grupo técnico ► Asignar ► Soporte
 ●
 ⋮

☐ Asignar tickets de Redes
 Añadir
 Categoría ► es ► Redes (Entidad raíz)
 Grupo técnico ► Asignar ► Soporte
 ●
 ⋮

☐ Asignar tickets de Sistemas
 Añadir
 Categoría ► es ► Sistemas (Entidad raíz)
 Grupo técnico ► Asignar ► Soporte
 ●
 ⋮

Se configuraron cuatro reglas de negocio en GLPI, una por cada categoría de incidencia. Cada regla sigue la misma lógica: si el ticket que llega tiene una categoría determinada, se le asigna automáticamente el grupo técnico Soporte. A continuación, se detalla cada regla:

Regla	Nombre	Criterio	Acción	Resultado
1	Asignar tickets de Hardware	Categoría = Hardware	Grupo técnico = Soporte	Los tickets de problemas con equipos físicos se asignan automáticamente
2	Asignar tickets de Software	Categoría = Software	Grupo técnico = Soporte	Los tickets de problemas con aplicaciones se asignan automáticamente
3	Asignar tickets de Redes	Categoría = Redes	Grupo técnico = Soporte	Los tickets de problemas de conectividad se asignan automáticamente
4	Asignar tickets de Sistemas	Categoría = Sistemas	Grupo técnico = Soporte	Los tickets de infraestructura y servidores se asignan automáticamente

Clasificación automática de urgencia:

```
# Palabras que indican urgencia muy alta
# Si el usuario escribe alguna de estas en el título o descripción,
# el ticket se crea con la máxima prioridad
PALABRAS_MUY_ALTA = [
    "caído", "caído", "no funciona", "sin acceso", "urgente",
    "crítico", "crítico", "no arranca", "bloqueado", "perdido",
    "virus", "hackeado", "no puedo trabajar", "parado"
]

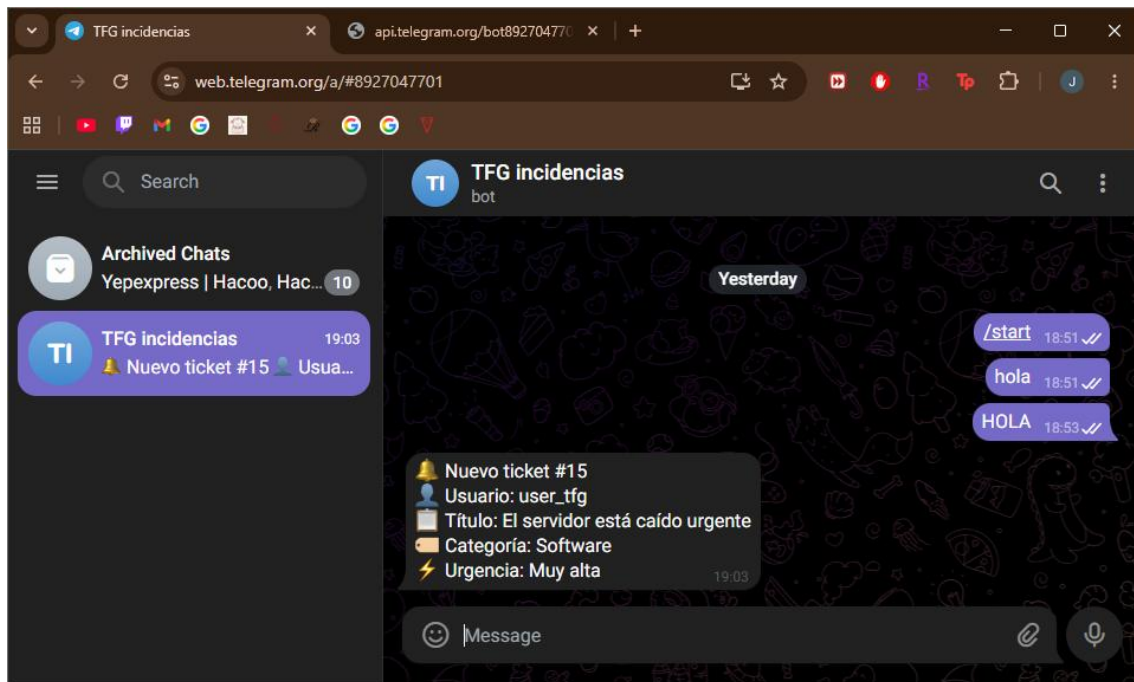
# Palabras que indican urgencia alta
PALABRAS_ALTA = [
    "fallo", "error", "no abre", "no carga", "lento",
    "problema grave", "no responde", "pantalla azul", "se cuelga"
]

# Palabras que indican urgencia media
PALABRAS_MEDIA = [
    "problema", "no puedo", "necesito ayuda", "no me deja",
    "no funciona bien", "a veces falla", "intermitente"
]

# Texto que se muestra al usuario y se manda por Telegram
# según el número de urgencia que devuelve GLPI
URGENCIA_TEXTO = {
    5: "Muy alta",
    4: "Alta",
    3: "Media",
    2: "Baja",
    1: "Muy baja"
}
```

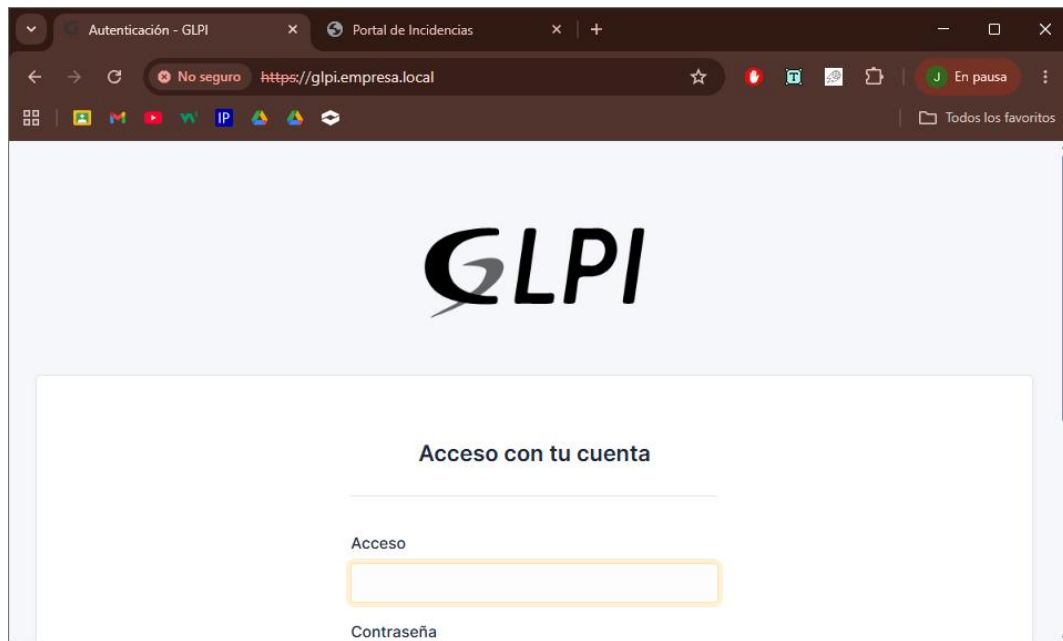
Se implementó la función `calcular_urgencia()` en el portal Flask que analiza el texto del ticket y asigna la urgencia sin intervención del usuario. Se probó creando un ticket con el título "El servidor está caído urgente" y el sistema asignó automáticamente urgencia Muy alta, verificándose tanto en el portal como en GLPI.

Bot de Telegram:

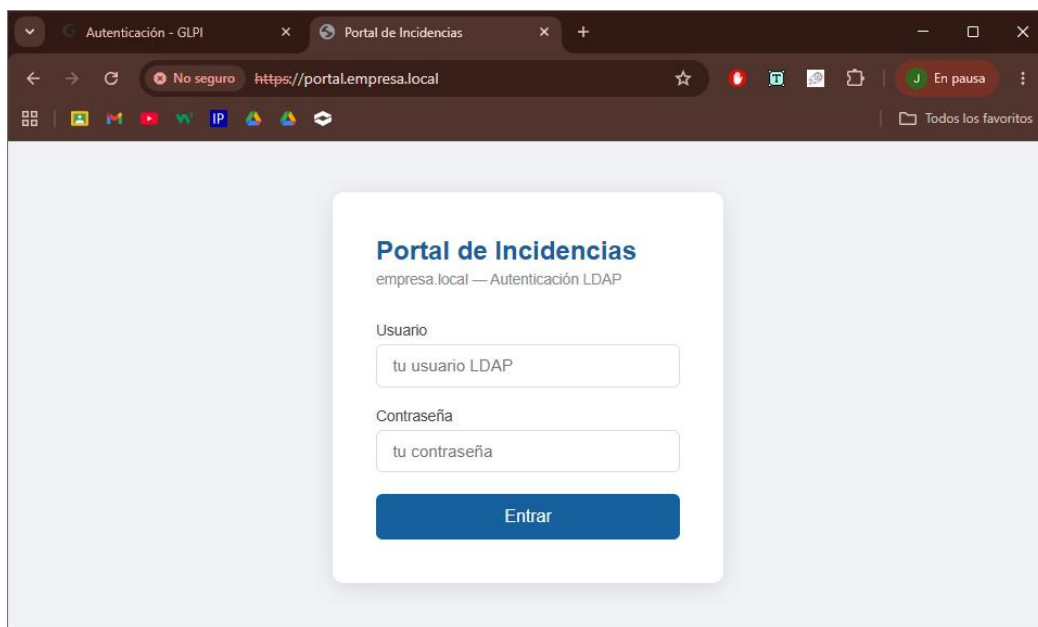


Se configuró el bot de Telegram mediante BotFather obteniendo el token de autenticación. Cada vez que se crea un ticket desde el portal Flask, el técnico recibe inmediatamente un mensaje en Telegram con el número de ticket, el usuario que lo abrió, el título, la categoría y la urgencia. Se verificó su funcionamiento creando varios tickets y comprobando que los mensajes llegaban correctamente en tiempo real. Nota: durante las pruebas realizadas en la red del centro educativo, las conexiones a `api.telegram.org` quedan bloqueadas por las políticas de la red. El bot funciona correctamente en redes sin restricciones.

Servidor DNS y HTTPS:



Se instaló BIND9 en el servidor LDAP y se configuró la zona empresa.local con los registros necesarios para resolver glpi.empresa.local y portal.empresa.local. Se generó un certificado SSL autofirmado con OpenSSL y se configuró Apache para servir GLPI sobre HTTPS, redirigiendo automáticamente el tráfico HTTP. El portal Flask se configuró detrás de un proxy inverso Apache para también servirse sobre HTTPS.



Se verificó que ambos dominios funcionan correctamente desde el PC anfitrión, accediendo a https://glpi.empresa.local para GLPI y https://portal.empresa.local para el portal de incidencias, sin necesidad de recordar ninguna IP.

5. CONCLUSIONES

Este proyecto me ha permitido construir un sistema de gestión de incidencias completo y funcional que va más allá de un ejercicio teórico, simulando de forma fiel lo que se encontraría en un entorno empresarial real.

El objetivo general se ha cumplido: GLPI está instalado, integrado con LDAP, protegido, automatizado y accesible desde un portal personalizado. Los trece objetivos específicos también se han alcanzado, incluyendo las dos mejoras propuestas por el tutor y las tres mejoras adicionales propias.

La parte más compleja fue la integración entre OpenLDAP y GLPI. Los problemas con el certificado TLS y la librería PHP me obligaron a investigar a fondo cómo funciona la autenticación TLS. La decisión final de usar el puerto 389 para la conexión interna está justificada porque la red 192.168.100.0/24 es una red completamente aislada, sin acceso desde el exterior.

El script de automatización y las reglas de asignación me han enseñado algo que considero clave para un técnico de sistemas: cualquier tarea repetitiva debe estar automatizada. El backup nocturno con subida automática a Dropbox y el enrutamiento automático de tickets son dos ejemplos concretos de cómo la automatización reduce errores humanos y libera tiempo del equipo técnico.

El portal Flask me enseñó a pensar en arquitectura en capas: GLPI es el backend, la API es el contrato entre sistemas y Flask es un frontend intercambiable.

La clasificación automática de urgencia por palabras clave demuestra que con muy poco código se puede añadir inteligencia a un sistema para reducir los errores humanos en la priorización de incidencias. El bot de Telegram es un ejemplo perfecto de integración entre sistemas: GLPI, Flask y la API de Telegram trabajan juntos de forma transparente. El servidor DNS con HTTPS cierra el círculo de profesionalización del sistema, eliminando las IPs y añadiendo cifrado en todas las comunicaciones.

En seguridad, el proyecto demuestra que la defensa en profundidad es el enfoque correcto: el WAF, las cabeceras HTTP, fail2ban, el hardening SSH y el HTTPS son capas independientes que se complementan.

6. LÍNEAS DE INVESTIGACIÓN FUTURAS

El proyecto desarrollado sienta una base sólida, pero abre la puerta a múltiples mejoras y extensiones que podrían abordarse en el futuro:

- Integrar el script de automatización con herramientas de monitorización reales como Prometheus o Grafana, de forma que cuando se detecte una caída de servicio o un uso elevado de recursos, se cree el ticket automáticamente en GLPI y se cierre cuando la alerta desaparezca, completando así el ciclo de vida completo de la incidencia sin intervención humana.
- Implementar replicación maestro-réplica en OpenLDAP para garantizar la alta disponibilidad del servicio de autenticación. Si el servidor principal cae, el secundario seguiría atendiendo las peticiones sin que los usuarios notaran ninguna interrupción.
- Sustituir el certificado SSL autofirmado por uno emitido por una CA interna usando Easy-RSA, eliminando el aviso de seguridad del navegador y acercando el sistema a los estándares de producción reales.
- Adaptar el sistema para conectarse a un Active Directory de Windows Server en lugar de OpenLDAP, lo que ampliaría enormemente la aplicabilidad del proyecto en entornos empresariales mixtos Windows/Linux.
- Desarrollar un panel de métricas que consuma la API REST de GLPI para mostrar en tiempo real estadísticas como el número de tickets abiertos, el tiempo medio de resolución y el rendimiento del equipo técnico.
- Ampliar el bot de Telegram para que el técnico pueda gestionar tickets directamente desde el móvil, por ejemplo escribiendo /cerrar 5 para cerrar el ticket número 5 sin necesidad de entrar en GLPI.
- Ampliar el sistema de clasificación automática de urgencia con inteligencia artificial para analizar el texto de forma más precisa y aprender de los tickets anteriores, mejorando la precisión de la clasificación con el tiempo.

7. BIBLIOGRAFÍA

Craig-Wood, N. (s/f). *Documentation*. Rclone. Recuperado el 21 de mayo de 2026, de <https://rclone.org/docs/>

Documentation Group. (s/f). *Documentation: Apache HTTP server - the Apache HTTP server project*. Apache.org. Recuperado el 21 de mayo de 2026, de <https://httpd.apache.org/docs/>

Help center GLPI. (s/f). Glpi-project.org. Recuperado el 21 de mayo de 2026, de <https://glpi-project.org/documentation/>

Home \ anthropic. (s/f). Anthropic.com. Recuperado el 21 de mayo de 2026, de <https://www.anthropic.com>

Lynis installation and usage guide - CISOfy. (s/f). Cisofy.com. Recuperado el 21 de mayo de 2026, de <https://cisofy.com/documentation/lynis/>

OWASP Top ten web application security risks. (s/f). Owasp.org. Recuperado el 21 de mayo de 2026, de <https://owasp.org/www-project-top-ten/>

Picón, K. [@kamilopicon1562]. (s/f). *Como instalar GLPI y Configuración de tickets - Marcos de Trabajo* [[Object Object]]. Youtube. Recuperado el 21 de mayo de 2026, de <https://www.youtube.com/watch?v=xNccE3CeWBg>

Sermersheim, J. (s/f). *RFC 4511: Lightweight Directory Access Protocol (LDAP): The protocol*. IETF Datatracker. Recuperado el 21 de mayo de 2026, de <https://tools.ietf.org/html/rfc4511>

solqubit [@solqubit]. (s/f). *Instalación completa de GLPI y agentes en Windows y Linux* [[Object Object]]. Youtube. Recuperado el 21 de mayo de 2026, de https://www.youtube.com/watch?v=0_xUwdTgPcY

User guide for release 7.2. (s/f). Virtualbox.org. Recuperado el 21 de mayo de 2026, de <https://www.virtualbox.org/manual/>

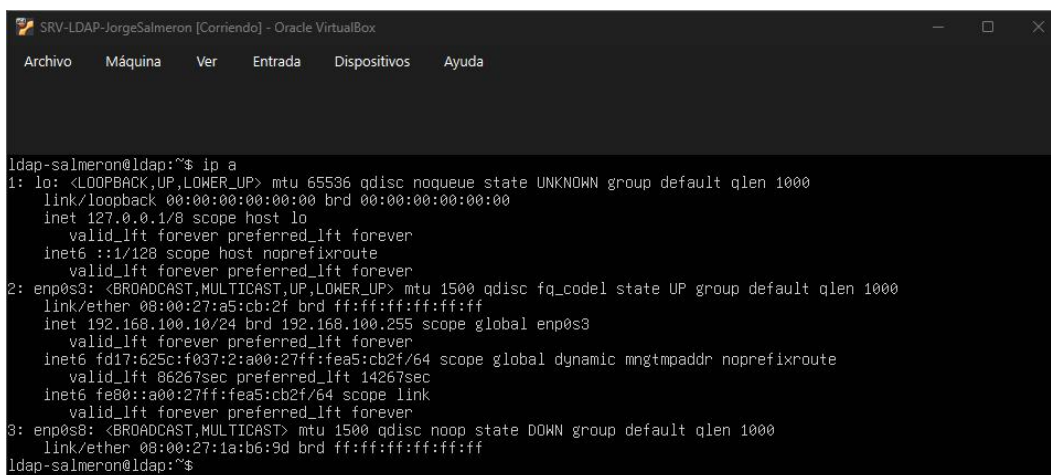
Welcome to flask — flask documentation (3.1.X). (s/f). Palletsprojects.com. Recuperado el 21 de mayo de 2026, de <https://flask.palletsprojects.com/>

(S/f). Openldap.org. Recuperado el 21 de mayo de 2026, de <https://www.openldap.org/doc/admin26/>

8. ANEXOS

Anexo A — Servidor LDAP

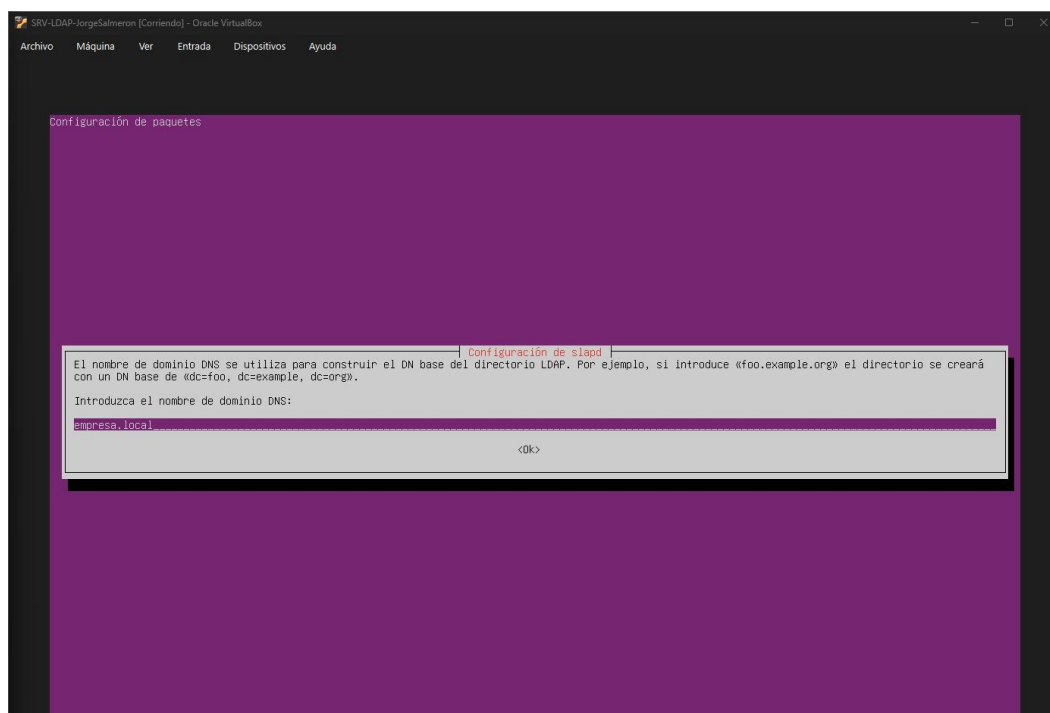
En este anexo se documenta el proceso de instalación y configuración del servidor OpenLDAP que actúa como directorio centralizado de usuarios del sistema.



```
SRV-LDAP-JorgeSalmeron [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

ldap-salmeron@ldap:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:a5:cb:2f brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.10/24 brd 192.168.100.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fd17:625c:f037:2:a00:27ff:fea5:cb2f/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86267sec preferred_lft 14267sec
    inet6 fe80::a00:27ff:fea5:cb2f/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 08:00:27:1a:b6:9d brd ff:ff:ff:ff:ff:ff
ldap-salmeron@ldap:~$
```

Configuración de la IP estática del servidor LDAP. Se asignó la dirección 192.168.100.10/24 en la interfaz de red interna para que el servidor GLPI pueda localizarlo siempre en la misma dirección.



```
SRV-LDAP-JorgeSalmeron [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

Configuración de paquetes

El nombre de dominio DNS se utiliza para construir el DN base del directorio LDAP. Por ejemplo, si introduce «foo.example.org» el directorio se creará con un DN base de «dc=foo, dc=example, dc=org».

Introduzca el nombre de dominio DNS:
empresa.local

<Ok>
```

Durante la instalación de OpenLDAP se configuró el dominio empresa.local, que define la raíz del directorio LDAP como dc=empresa,dc=local.

```
jorge@ldap-salmeron:~$ ldapsearch -x -b "dc=empresa,dc=local" "(objectClass=inetOrgPerson)" cn
# extended LDIF
#
# LDAPv3
# base <dc=empresa,dc=local> with scope subtree
# filter: (objectClass=inetOrgPerson)
# requesting: cn
#
# admin_glpi, usuarios, empresa.local
dn: cn=admin_glpi,ou=usuarios,dc=empresa,dc=local
cn: admin_glpi
# tecnico1, usuarios, empresa.local
dn: cn=tecnico1,ou=usuarios,dc=empresa,dc=local
cn: tecnico1
# user_tfg, usuarios, empresa.local
dn: cn=user_tfg,ou=usuarios,dc=empresa,dc=local
cn: user_tfg
# search result
search: 2
result: 0 Success
# numResponses: 4
# numEntries: 3
jorge@ldap-salmeron:~$
```

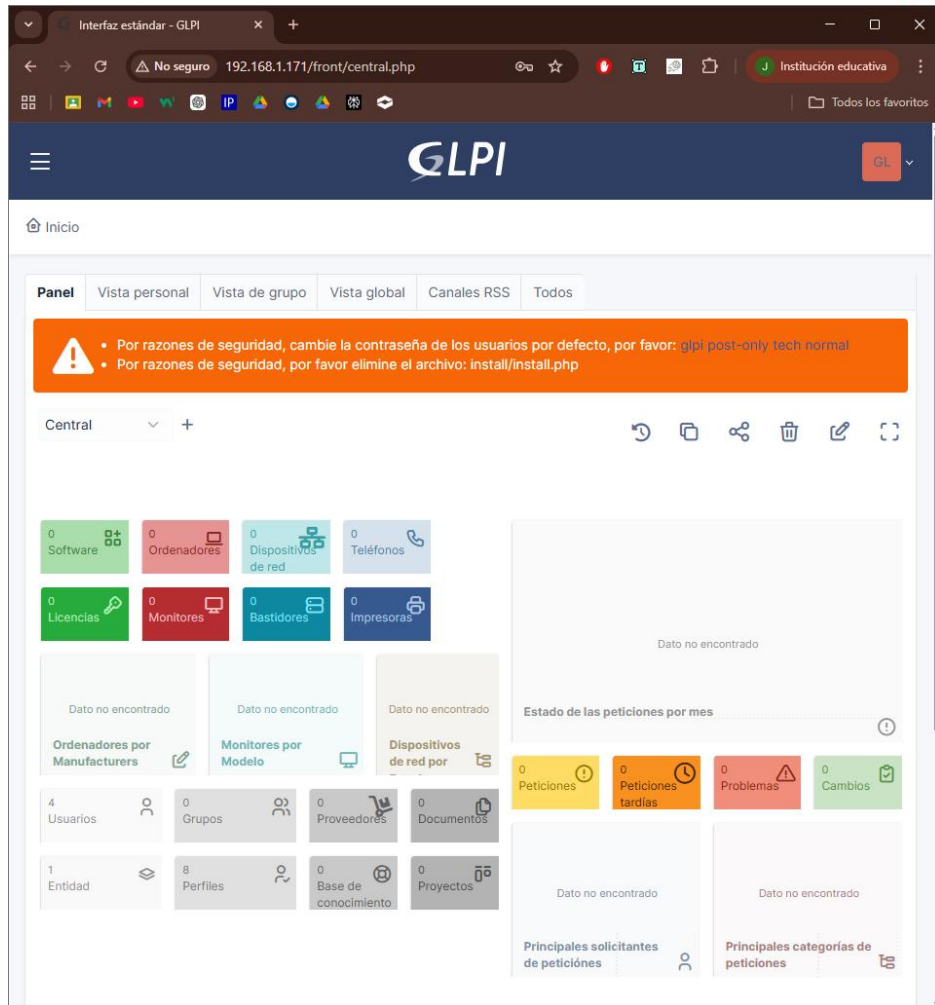
Verificación final del directorio LDAP mostrando los tres usuarios creados: admin_glpi, tecnico1 y user_tfg, confirmando que el directorio funciona correctamente.

Anexo B — Instalación GLPI

En este anexo se documenta el proceso completo de instalación de GLPI sobre el servidor web Apache2 con MariaDB y PHP.



Pantalla del asistente de instalación web de GLPI mostrando que la conexión con la base de datos MariaDB se estableció correctamente con el usuario user_glpi.



Primera pantalla tras completar la instalación, confirmando que GLPI está operativo y accesible desde el navegador.

```
jorge-glpi@srv-glpi:~$ ls /var/www/glpi/
ajax          caldav.php    css_compiled  INSTALL.md    marketplace   resources      SUPPORT.md
apirest.md    CHANGELOG.md files          js            pics          SECURITY.md    templates
apirest.php   config        front         lib           plugins       sound          vendor
apixmlrpc.php CONTRIBUTING.md inc           LICENSE       public        src            version
bin           css           index.php     locales       README.md    status.php
```

Verificación de que la carpeta de instalación fue eliminada correctamente por seguridad tras completar el proceso.

Anexo C — Integración LDAP

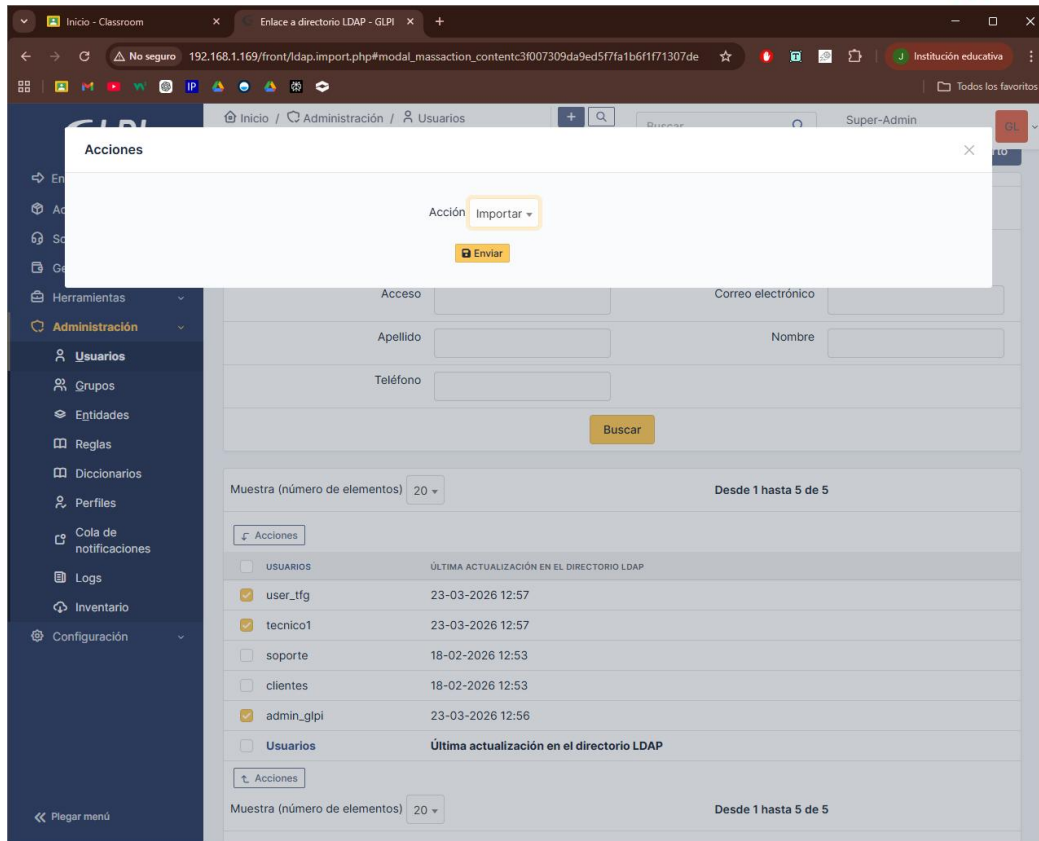
En este anexo se documenta el proceso de conexión entre el servidor GLPI y el directorio OpenLDAP para la autenticación centralizada de usuarios.

Configuración del directorio LDAP dentro de GLPI mostrando todos los parámetros: servidor 192.168.100.10, puerto 389, BaseDN dc=empresa,dc=local y RootDN cn=admin,dc=empresa,dc=local.

```
jorge-glpi@srv-glpi:~$ ping -c 3 192.168.100.10
PING 192.168.100.10 (192.168.100.10) 56(84) bytes of data.
64 bytes from 192.168.100.10: icmp_seq=1 ttl=64 time=0.496 ms
64 bytes from 192.168.100.10: icmp_seq=2 ttl=64 time=0.329 ms
64 bytes from 192.168.100.10: icmp_seq=3 ttl=64 time=0.267 ms

--- 192.168.100.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2115ms
rtt min/avg/max/mdev = 0.267/0.364/0.496/0.096 ms
jorge-glpi@srv-glpi:~$
```

Resultado de la prueba de conexión dentro de GLPI mostrando que el servidor LDAP responde correctamente y devuelve la lista de usuarios del directorio.



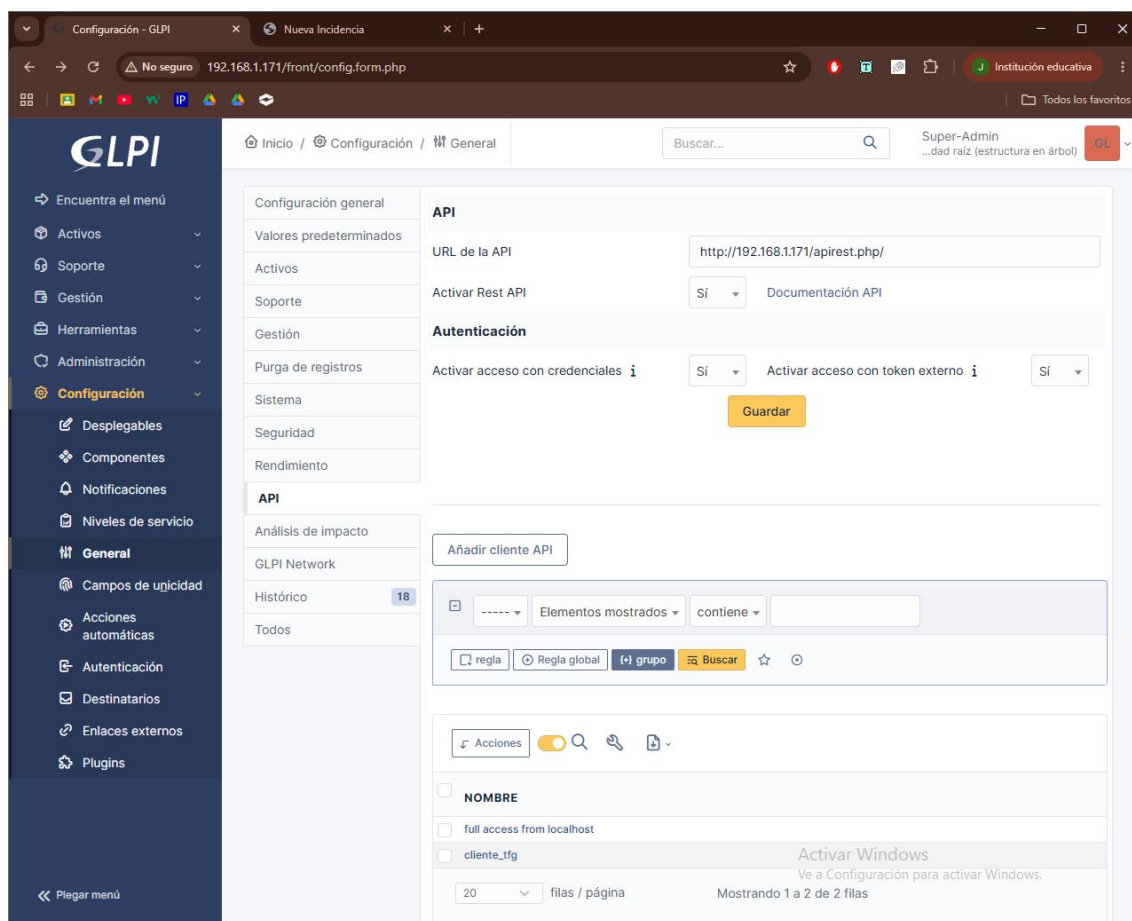
Proceso de importación de los usuarios del directorio LDAP a GLPI, mostrando los usuarios admin_glpi, tecnico1 y user_tfg disponibles para importar.

ACCESO	APELLIDO	CORREOS ELECTRÓNICOS	TÉLEFONO	UBICACIONES	ACTIVO
☐ S	admin_glpi	sistemas			Sí
☐ GL	glpi				Sí
☐ S	glpi-system	Support			Sí
☐ NO	normal				Sí
☐ PO	post-only				Sí
☐ TE	tech				Sí
☐ L	tecnico1	Lopez			Sí
☐ G	user_tfg	Gomez			Sí

Resultado final mostrando los tres usuarios del directorio LDAP correctamente importados en GLPI con sus perfiles asignados.

Anexo D — API REST y Zero-Touch Ticketing

En este anexo se documenta la activación de la API REST de GLPI y el desarrollo del script Python de automatización de tickets.



Pantalla de Configuración → General → API mostrando la API REST activada y el App Token generado para el cliente cliente_tfg.

```
jorge-glp@srv-glp: ~
jorge-glp@srv-glp:~$ curl -s -X GET "http://192.168.1.169/apiest.php/initSession" \
-H "Authorization: user_token GWgonI0giWhAO2FA4GGLwpTbCwkkxtfl8zyAKTYa" \
-H "App-Token: 8QvMLgG9ilq7UB2BhU7Iodl7u6fhfIMd3taADzel"
{"session_token": "152p0upq3s9lodvcksgvv4b9o4"}jorge-glp@srv-glp:~$
```

Prueba desde la terminal del servidor mostrando que la API responde correctamente devolviendo un session_token válido ante una petición de inicio de sesión.

```
import requests, sys

# Dirección de GLPI y claves de autenticación de la API
GLPI_URL = "http://192.168.1.169/apirest.php"
APP_TOKEN = "8QvMLgG9ilq7UB2BhU7Iodl7u6fhfIMd3taADzel"
USER_TOKEN = "GWgonI0giWhAO2FA4GGLwpTbCwkkxtf18zyAKTYa"

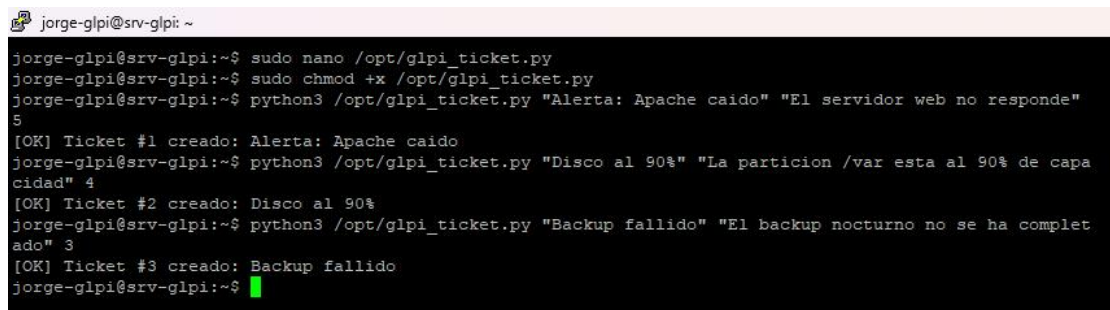
def iniciar_sesion():
    # Inicia sesión en la API y obtiene un token de sesión temporal
    r = requests.get(f"{GLPI_URL}/initSession",
        headers={"Authorization": f"user_token {USER_TOKEN}", "App-Token":
APP_TOKEN})
    r.raise_for_status()
    return r.json()["session_token"]

def cerrar_sesion(st):
    # Cierra la sesión para liberar recursos
    requests.get(f"{GLPI_URL}/killSession",
        headers={"Session-Token": st, "App-Token": APP_TOKEN})

def crear_ticket(titulo, descripcion, urgencia=3):
    # Urgencia: 1=Muy baja 2=Baja 3=Media 4=Alta 5=Muy alta
    st = iniciar_sesion()
    try:
        r = requests.post(f"{GLPI_URL}/Ticket",
            headers={"Session-Token": st, "App-Token": APP_TOKEN,
                "Content-Type": "application/json"},
            json={"input": {
                "name": titulo, #Título del ticket
                "content": descripcion, #Descripción del problema
                "urgency": int(urgencia), # Nivel de urgencia
                "type": 1 # 1 = Incidencia
            }})
        tid = r.json().get("id", "ERROR")
        print(f"[OK] Ticket #{tid} creado: {titulo}")
    finally:
        cerrar_sesion(st) # Se cierra siempre aunque haya error

# Arranca desde la terminal con: python3 glpi_ticket.py "Título" "Descripción"
urgencia
if __name__ == "__main__":
    titulo = sys.argv[1] if len(sys.argv) > 1 else "Alerta automatica"
    desc = sys.argv[2] if len(sys.argv) > 2 else "Incidencia detectada
automaticamente"
    urg = int(sys.argv[3]) if len(sys.argv) > 3 else 3
    crear_ticket(titulo, desc, urg)
```

Contenido del script Python de automatización con todos sus comentarios explicativos.



```
jorge-glpi@srv-glpi: ~
jorge-glpi@srv-glpi:~$ sudo nano /opt/glpi_ticket.py
jorge-glpi@srv-glpi:~$ sudo chmod +x /opt/glpi_ticket.py
jorge-glpi@srv-glpi:~$ python3 /opt/glpi_ticket.py "Alerta: Apache caido" "El servidor web no responde"
5
[OK] Ticket #1 creado: Alerta: Apache caido
jorge-glpi@srv-glpi:~$ python3 /opt/glpi_ticket.py "Disco al 90%" "La particion /var esta al 90% de capa
cidad" 4
[OK] Ticket #2 creado: Disco al 90%
jorge-glpi@srv-glpi:~$ python3 /opt/glpi_ticket.py "Backup fallido" "El backup nocturno no se ha complet
ado" 3
[OK] Ticket #3 creado: Backup fallido
jorge-glpi@srv-glpi:~$
```

Ejecución del script creando tres tickets de prueba con diferentes niveles de urgencia, mostrando la confirmación de creación de cada uno.

Verificación en la interfaz de GLPI de que los tres tickets creados por el script aparecen correctamente con sus datos y el estado Nuevo.

Anexo E — Seguridad y Hardening

En este anexo se documenta todo el proceso de securización del sistema: WAF, cabeceras HTTP, auditorías y herramientas de protección.

```
jorge-glpi@srv-glpi:~$ sudo apt install git -y
sudo git clone https://github.com/coreruleset/coreruleset /etc/apache2/modsecurity-crs
sudo cp /etc/apache2/modsecurity-crs/crs-setup.conf.example /etc/apache2/modsecurity-crs/crs-setup.conf
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
git ya está en su versión más reciente (1:2.43.0-lubuntu7.3).
fijado git como instalado manualmente.
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
cpp cpp-13 cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu fakeroot g++ g++-13 g++-13-x86-64-linux-gnu
g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu
javascript-common libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan8
libatomic1 libccol-0 libexpat1-dev libfakeroot libfile-fcntllock-perl libgcc-13-dev libhwasan0
libisl23 libitm1 libjs-jquery libjs-sphinxdoc libjs-underscore liblsan0 libmpc3 libquadmath0
libstdc++-13-dev libtsan2 libubsan1 lto-disabled-list zlib1g-dev
Utilice «sudo apt autoremove» para eliminarlos.
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Cloning into '/etc/apache2/modsecurity-crs'...
remote: Enumerating objects: 53549, done.
remote: Counting objects: 100% (55/55), done.
remote: Compressing objects: 100% (41/41), done.
remote: Total 53549 (delta 29), reused 33 (delta 14), pack-reused 53494 (from 3)
Receiving objects: 100% (53549/53549), 13.64 MiB | 17.76 MiB/s, done.
Resolving deltas: 100% (38124/38124), done.
jorge-glpi@srv-glpi:~$
```

Verificación de que el módulo ModSecurity está cargado correctamente en Apache.

```
jorge-glpi@srv-glpi: ~
jorge-glpi@srv-glpi:~$ sudo systemctl restart apache2
jorge-glpi@srv-glpi:~$ curl -I "http://192.168.1.169/?id=1'OR'1='1"
HTTP/1.1 403 Forbidden
Date: Mon, 11 May 2026 19:41:37 GMT
Server: Apache/2.4.58 (Ubuntu)
Content-Type: text/html; charset=iso-8859-1
jorge-glpi@srv-glpi:~$
```

Prueba de ataque simulado mostrando que ModSecurity bloquea correctamente una inyección SQL devolviendo HTTP 403 Forbidden.

```
jorge-glpi@srv-glpi: /opt/portal_tfg

=====

Lynis security scan details:

Hardening index : 63 [#####          ]
Tests performed : 269
Plugins enabled : 1

Components:
- Firewall           [V]
- Malware scanner    [V]

Scan mode:
Normal [V]  Forensics [ ]  Integration [ ]  Pentest [ ]

Lynis modules:
- Compliance status  [?]
- Security audit     [V]
- Vulnerability scan [V]

Files:
- Test and debug information : /var/log/lynis.log
- Report data                : /var/log/lynis-report.dat
```

Resultado de la auditoría de seguridad del sistema operativo con Lynis mostrando la puntuación final de 63 sobre 100 tras aplicar las correcciones recomendadas.

```
jorge-glpi@srv-glpi: /opt/portal_tfg

● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-05-20 18:07:21 UTC; 1h 46min ago
     Docs: man:fail2ban(1)
   Main PID: 717 (fail2ban-server)
    Tasks: 5 (limit: 4596)
   Memory: 40.0M (peak: 40.5M)
      CPU: 4.110s
   CGroup: /system.slice/fail2ban.service
           └─717 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

may 20 18:07:21 srv-glpi systemd[1]: Started fail2ban.service - Fail2Ban Service.
may 20 18:07:22 srv-glpi fail2ban-server[717]: 2026-05-20 18:07:22,449 fail2ban.configreader [717]: W
may 20 18:07:23 srv-glpi fail2ban-server[717]: Server ready
```

Verificación de que fail2ban está instalado y activo, protegiendo el servidor contra ataques de fuerza bruta.

Anexo F — Portal Flask

En este anexo se documenta el desarrollo y funcionamiento del portal web personalizado para la creación de incidencias. (Este script es largo, está dividido en 2).

Parte 1 – Configuración y funciones:

```
from flask import Flask, render_template, request, redirect, url_for, session
from ldap3 import Server, Connection, ALL
import requests

app = Flask(__name__)

app.secret_key = "tfg_jorgesalmeron_2025_secreto" #Clave para gestionar sesiones

# Configuración de GLPI, LDAP y Telegram

GLPI_URL = "https://192.168.1.171/apirest.php"
APP_TOKEN = "8QvMLgG9ilq7UB2BhU7Iodl7u6fhfIMd3taADzel"
USER_TOKEN = "GWgonI0giWhAO2FA4GGLwpTbCwkkxtf18zyAKTYa"
LDAP_HOST = "192.168.100.10"
LDAP_PORT = 389
LDAP_BASE = "dc=empresa,dc=local"
TELEGRAM_TOKEN = "8927047701:AAGpiPiOeiCZzJqxzjWlelqlA0hUBuPs00A"
TELEGRAM_CHAT_ID = "1342703201"

# IDs de las categorías en GLPI

CATEGORIAS = {"Hardware": 3, "Software": 4, "Red y conectividad": 2,
              "Accesos y permisos": 1, "Otro": 0}

# Listas de palabras clave para calcular la urgencia automáticamente

PALABRAS_MUY_ALTA = ["caído", "caido", "no funciona", "sin acceso", "urgente",
                    "crítico", "no arranca", "bloqueado", "virus", "parado"]

PALABRAS_ALTA = ["fallo", "error", "no abre", "lento", "no responde", "se cuelga"]

PALABRAS_MEDIA = ["problema", "no puedo", "necesito ayuda", "a veces falla"]

URGENCIA_TEXTO = {5: "Muy alta", 4: "Alta", 3: "Media", 2: "Baja", 1: "Muy baja"}

def calcular_urgencia(titulo, descripcion):
    # Analiza el texto y asigna urgencia según palabras clave encontradas
    texto = (titulo + " " + descripcion).lower()
    for p in PALABRAS_MUY_ALTA:
```

```

        if p in texto: return 5
    for p in PALABRAS_ALTA:
        if p in texto: return 4
    for p in PALABRAS_MEDIA:
        if p in texto: return 3
    return 2 # Baja por defecto

def enviar_telegram(ticket_id, titulo, urgencia, categoria, usuario):
    # Manda un aviso al técnico por Telegram con los datos del ticket
    texto = (f"🔔 Nuevo ticket #{ticket_id}\n👤 Usuario: {usuario}\n"
             f"📄 Título: {titulo}\n📁 Categoría: {categoria}\n"
             f"⚡ Urgencia: {URGENCIA_TEXTO.get(urgencia, urgencia)}")
    try:
        requests.post(f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage",
                      json={"chat_id": TELEGRAM_CHAT_ID, "text": texto})
    except Exception as e:
        print(f"Error Telegram: {e}")

def autenticar_ldap(usuario, password):
    # Valida las credenciales del usuario contra el directorio LDAP
    try:
        conn = Connection(Server(LDAP_HOST, port=LDAP_PORT, get_info=ALL),
                          user=f"cn={usuario},ou=usuarios,{LDAP_BASE}",
                          password=password, auto_bind=True)
        return conn.bound
    except Exception as e:
        print(f"Error LDAP: {e}"); return False

def crear_ticket_glpi(titulo, descripcion, urgencia, categoria):
    # Inicia sesión en la API, crea el ticket y cierra la sesión
    r = requests.get(f"{GLPI_URL}/initSession", verify=False,
                    headers={"Authorization": f"user_token {USER_TOKEN}",
                             "App-Token": APP_TOKEN})
    st = r.json()["session_token"]
    r2 = requests.post(f"{GLPI_URL}/Ticket", verify=False,
                     headers={"Session-Token": st, "App-Token": APP_TOKEN,
                              "Content-Type": "application/json"},
                     json={"input": {"name": titulo, "content": descripcion,

```



```

        "urgency": int(urgencia), "type": 1,
        "itilcategories_id":
CATEGORIAS.get(categoria,0) })
    requests.get(f"{GLPI_URL}/killSession", verify=False,
                  headers={"Session-Token": st, "App-Token": APP_TOKEN})
    resultado = r2.json()
    return resultado[0].get("id") if isinstance(resultado, list) else
resultado.get("id")

```

Parte 2 – Rutas:

```

@app.route("/", methods=["GET", "POST"])
def login():
    error = None
    if request.method == "POST":
        u,p = request.form["usuario"], request.form["password"]
        if autenticar_ldap(u,p):
            session["usuario"] = u
            return redirect(url_for("ticket"))
        error = "Usuario o contraseña incorrectos"
    return render_template("login.html", error=error)

@app.route("/ticket", methods=["GET", "POST"])
def ticket():
    if "usuario" not in session:
        return redirect(url_for("login"))
    ticket_id, urgencia_asignada = None, None
    if request.method == "POST":
        titulo = request.form["titulo"]
        descripcion = request.form["descripcion"]
        categoria = request.form.get("categoria", "Otro")
        urgencia_asignada = calcular_urgencia(titulo, descripcion)
        ticket_id =
crear_ticket_glpi(titulo, descripcion, urgencia_asignada, categoria)
        if ticket_id:
            enviar_telegram(ticket_id, titulo, urgencia_asignada, categoria, session["usuario"])
    return render_template("ticket.html", usuario=session["usuario"],
                           ticket_id=ticket_id, urgencia=urgencia_asignada)

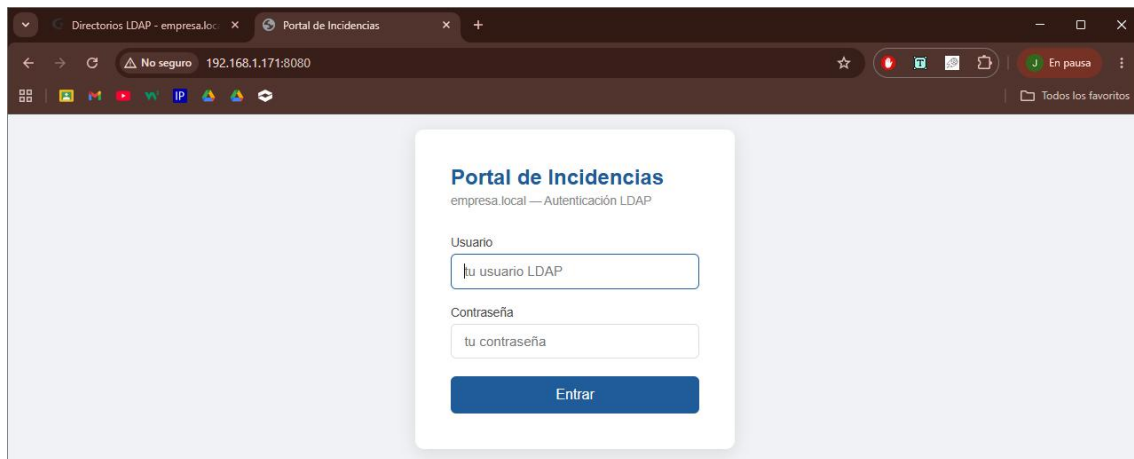
@app.route("/logout")

```

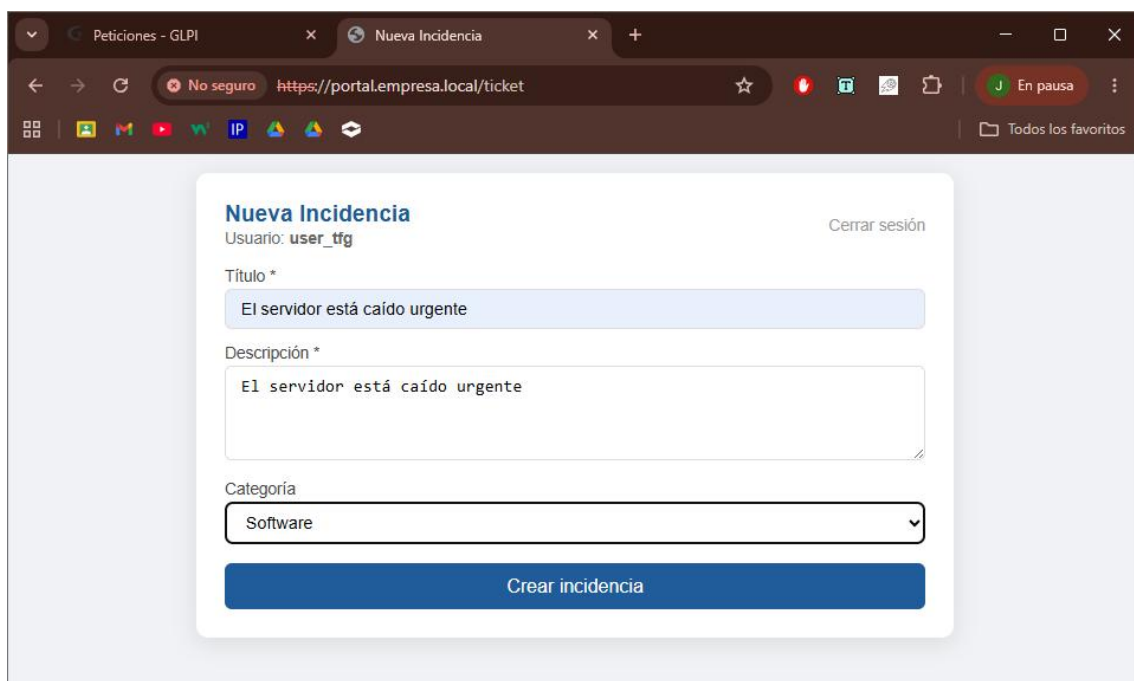
```
def logout():
    session.clear()
    return redirect(url_for("login"))

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080, debug=False)
```

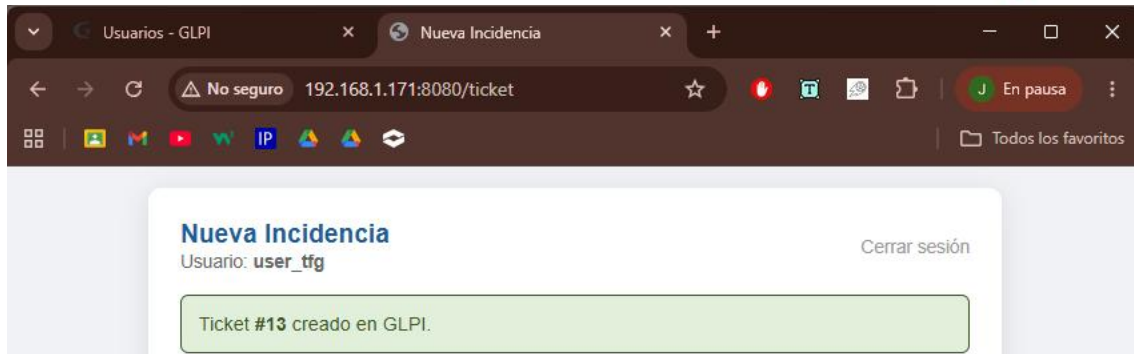
Contenido del archivo principal del portal Flask con todos sus comentarios explicativos.



Pantalla de inicio del portal accesible desde el navegador mostrando el formulario de autenticación LDAP.



Pantalla del formulario de creación de incidencias con los campos título, descripción y categoría rellenos, antes de enviarlo.



Mensaje de confirmación que aparece tras enviar el formulario, mostrando el número de ticket asignado por GLPI.

							SOLICITANTE	ASIGNADA		
							-	A -		
☐	ID	TÍTULO	ESTADO	ÚLTIMA ACTUALIZACIÓN	FECHA DE APERTURA	PRIORIDAD	SOLICITANTE	TÉCNICO	CATEGORÍA	
☐	12	Falta de puertos USB	En curso (asignada)	20-05-2026 20:21	20-05-2026 20:21	Media			Hardware	

Verificación en GLPI del ticket creado desde el portal, mostrando la categoría correcta y el grupo Soporte asignado automáticamente por las reglas de negocio.

Anexo G — Backup y Asignación automática

En este anexo se documenta el sistema de copias de seguridad automáticas de la base de datos y su integración con Dropbox mediante rclone.

```
#!/bin/bash

# Configuración: base de datos, carpeta de destino y días de retención
DB_USER="root"
DB_NAME="glpi"
BACKUP_DIR="/opt/backups_glpi"
FECHA=$(date +%Y-%m-%d_%H-%M) # Fecha y hora para el nombre del
archivo
ARCHIVO="$BACKUP_DIR/glpi_backup_$FECHA.sql.gz"
DIAS_RETENCION=7 # Días que se conservan las copias

# Crea la carpeta de backups si no existe
mkdir -p $BACKUP_DIR

# Vuelca la base de datos completa y la comprime con gzip
mysqldump -u $DB_USER $DB_NAME | gzip > $ARCHIVO

# Comprueba si el backup se creó correctamente ($? = 0 si hubo éxito)
if [ $? -eq 0 ]; then
    echo "[$FECHA] Backup creado: $ARCHIVO"
else
    echo "[$FECHA] ERROR: el backup fallo"
    exit 1
fi

# Borra automáticamente los backups con más de 7 días de antigüedad
find $BACKUP_DIR -name "*.sql.gz" -mtime +$DIAS_RETENCION -delete
echo "[$FECHA] Limpieza completada"

# Sube todos los backups a Dropbox mediante rclone
rclone copy /opt/backups_glpi/ dropbox:backups_glpi
echo "[$FECHA] Backup subido a Dropbox correctamente"
```

Contenido del script de backup con todos sus comentarios explicativos mostrando cada parte del proceso.

```
jorge-glpi@srv-glpi: ~
jorge-glpi@srv-glpi:~$ sudo mkdir -p /root/.config/rclone
sudo cp ~/.config/rclone/rclone.conf /root/.config/rclone/rclone.conf
jorge-glpi@srv-glpi:~$ sudo /opt/backup_glpi.sh
[2026-05-21_19-49] Backup creado: /opt/backups_glpi/glpi_backup_2026-05-21_19-49.sql.gz
[2026-05-21_19-49] Limpieza completada
[2026-05-21_19-49] Backup subido a Dropbox correctamente
jorge-glpi@srv-glpi:~$ ^C
jorge-glpi@srv-glpi:~$
```

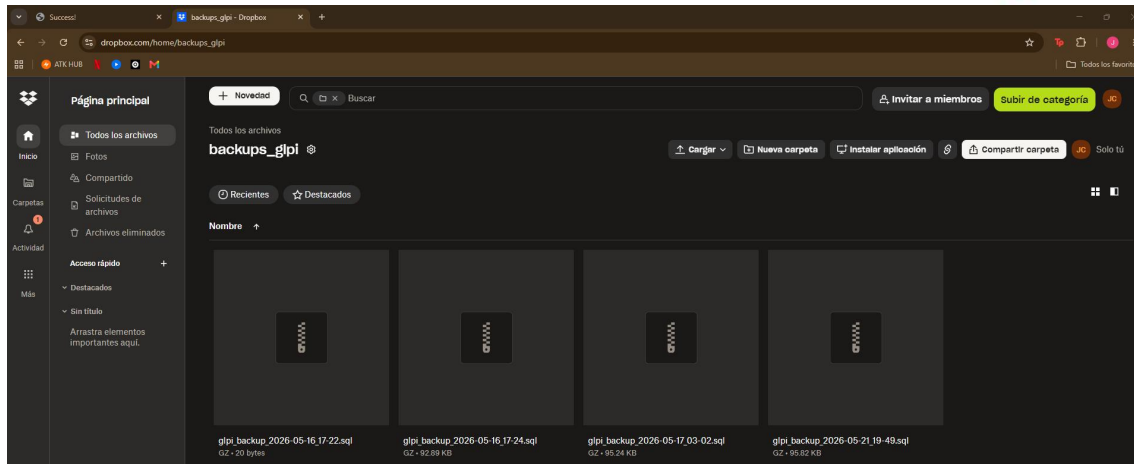
Ejecución manual del script mostrando que el backup se crea localmente y se sube a Dropbox sin errores.

```
jorge-glpi@srv-glpi: /opt/portal_tfg
jorge-glpi@srv-glpi:/opt/portal_tfg$ ls -lh /opt/backups_glpi/
total 196K
-rw-r--r-- 1 root root 20 may 16 17:22 glpi_backup_2026-05-16_17-22.sql.gz
-rw-r--r-- 1 root root 93K may 16 17:24 glpi_backup_2026-05-16_17-24.sql.gz
-rw-r--r-- 1 root root 96K may 17 03:02 glpi_backup_2026-05-17_03-02.sql.gz
jorge-glpi@srv-glpi:/opt/portal_tfg$
```

Listado de la carpeta de backups locales mostrando los archivos comprimidos con su tamaño y fecha de creación.

```
jorge-glpi@srv-glpi: /opt/portal_tfg
jorge-glpi@srv-glpi:/opt/portal_tfg$ sudo crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
0 2 * * * /opt/backup_glpi.sh >> /var/log/backup_glpi.log 2>&1
jorge-glpi@srv-glpi:/opt/portal_tfg$
```

Verificación de la tarea programada en cron que ejecutará el script automáticamente cada noche a las 2:00.



Verificación en la interfaz web de Dropbox de que los archivos de backup están correctamente subidos a la nube.

Anexo H — Reglas de asignación automática de tickets

En este anexo se documenta la configuración de las reglas de negocio en GLPI para la asignación automática de tickets según la categoría del departamento.



Las cuatro categorías creadas en GLPI: Hardware, Software, Redes y Sistemas, que se usan como criterio en las reglas de asignación.

Inicio / Administración / Reglas / Reglas de nego...

Regla - Asignar tickets de Hardware Acciones

Regla

Criteria

Acciones

Histórico 1

Todos

Nuevo elemento - Criterios

Criterios

Criteria - Categoría

es

Hardware i +

+ Añadir

Añadir un nuevo criterio

Criterios	Condición	Motivo
-----------	-----------	--------

Inicio / Administración / Reglas / Reglas de nego...

Regla - Asignar tickets de Hardware Acciones

Regla

Criteria 1

Acciones

Histórico 3

Todos

Nuevo elemento - Acción

Acción

Grupo técnico

Asignar

Soporte i +

+ Añadir

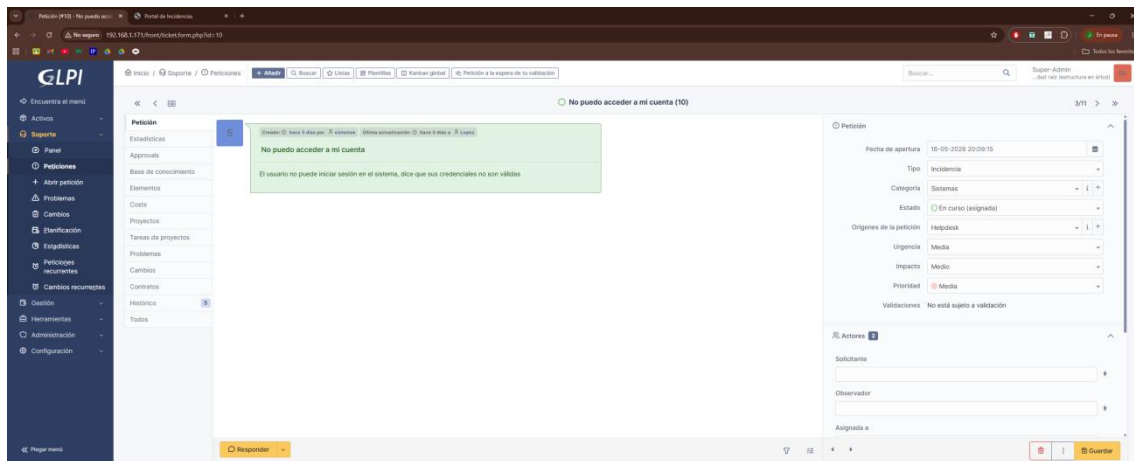
Añadir una acción nueva

Campos	Tipo de acción	Valor
--------	----------------	-------

Vista detallada de una de las reglas mostrando el criterio (categoría = Hardware) y la acción (asignar grupo técnico = Soporte).

☐ Asignar tickets de Hardware	Asigna tickets de Hardware al grupo Soporte	Añadir	Categoría ▶ es ▶ Hardware (Entidad raíz)	Grupo técnico ▶ Asignar ▶ Soporte	●	⋮
☐ Asignar tickets de Software		Añadir	Categoría ▶ es ▶ Software (Entidad raíz)	Grupo técnico ▶ Asignar ▶ Soporte	●	⋮
☐ Asignar tickets de Redes		Añadir	Categoría ▶ es ▶ Redes (Entidad raíz)	Grupo técnico ▶ Asignar ▶ Soporte	●	⋮
☐ Asignar tickets de Sistemas		Añadir	Categoría ▶ es ▶ Sistemas (Entidad raíz)	Grupo técnico ▶ Asignar ▶ Soporte	●	⋮

Pantalla de Administración → Reglas → Reglas de negocio para tickets mostrando las cuatro reglas activas.



Verificación de que un ticket creado con categoría Hardware aparece en GLPI con el grupo Soporte asignado automáticamente sin intervención manual.

Anexo I — DNS con BIND9

En este anexo se documenta la instalación y configuración del servidor DNS BIND9 para la resolución de nombres de dominio en la red interna.

```
jorge@srv-ldap: ~
jorge@srv-ldap:~$ systemctl status bind9
● named.service - BIND Domain Name Server
   Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-05-27 17:15:00 UTC; 5h 17min ago
     Docs: man:named(8)
   Main PID: 10618 (named)
    Status: "running"
   Tasks: 5 (limit: 2263)
  Memory: 23.7M (peak: 24.0M)
     CPU: 101ms
    CGroup: /system.slice/named.service
            └─10618 /usr/sbin/named -f -u bind
```

Verificación de que el servidor DNS BIND9 está instalado y activo en el servidor LDAP.

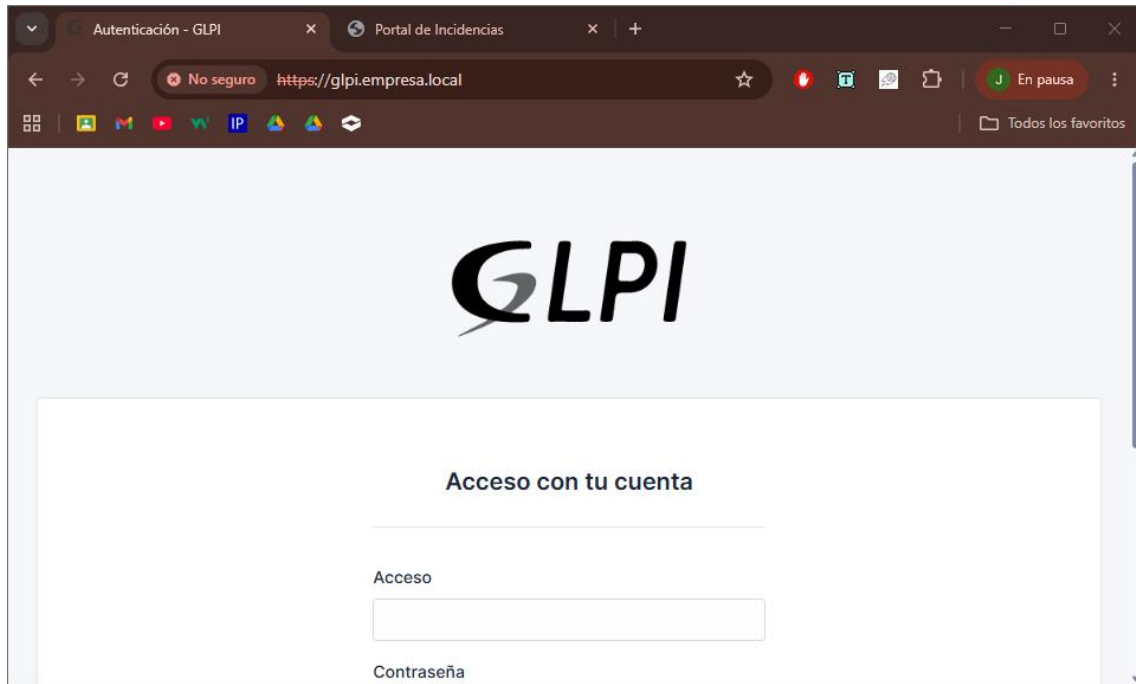
```
jorge@srv-ldap: ~
jorge@srv-ldap:~$ cat /etc/bind/db.empresa.local
$TTL 604800
@      IN      SOA      srv-ldap.empresa.local. admin.empresa.local. (
                                2           ; Serial
                                604800      ; Refresh
                                86400       ; Retry
                                2419200    ; Expire
                                604800 )   ; Negative Cache TTL
;
@      IN      NS       srv-ldap.empresa.local.
@      IN      A        192.168.100.10
srv-ldap      IN      A        192.168.100.10
srv-glpi      IN      A        192.168.100.20
glpi          IN      A        192.168.100.20
portal        IN      A        192.168.1.171
jorge@srv-ldap:~$
```

Archivo de zona DNS mostrando los registros configurados: srv-ldap, srv-glpi, glpi y portal apuntando a sus respectivas IPs. Es como una agenda de contactos, pero para servidores. En lugar de buscar una IP a mano, el sistema consulta este archivo y traduce automáticamente el nombre a la IP correspondiente. Sin este archivo, cuando escribes glpi.empresa.local en el navegador, el sistema no sabría a qué servidor ir.

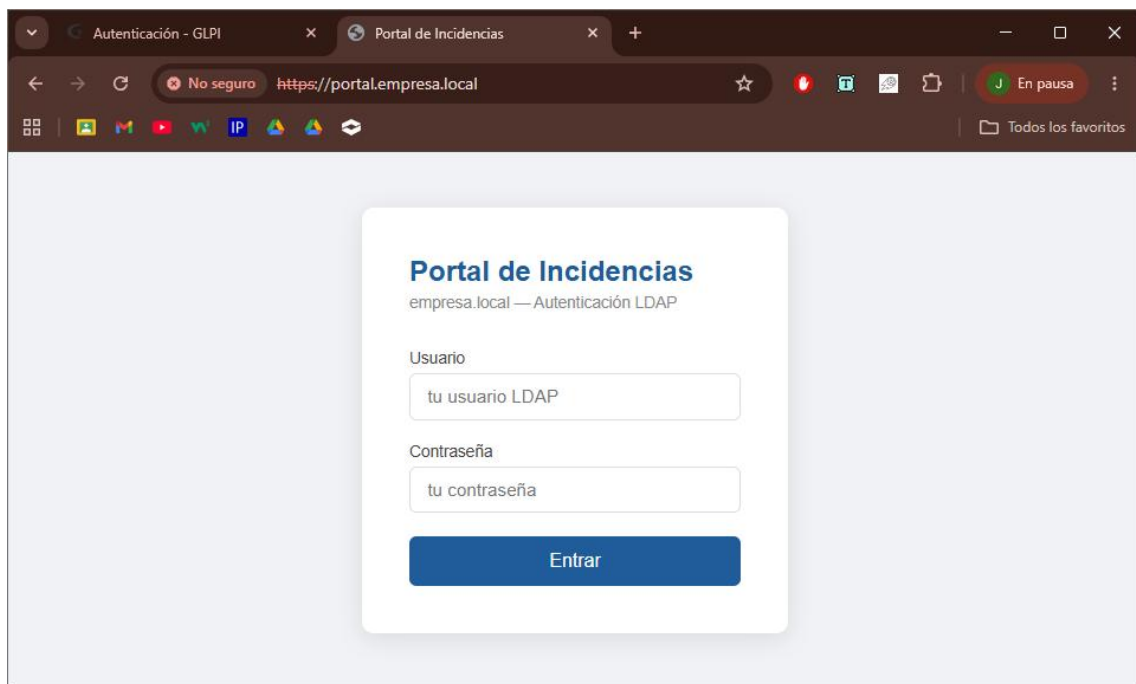
```
jorge-glpi@srv-glpi: /opt/portal_tfg
jorge-glpi@srv-glpi:/opt/portal_tfg$ nslookup glpi.empresa.local
Server:          127.0.0.53
Address:         127.0.0.53#53

Non-authoritative answer:
Name:   glpi.empresa.local
Address: 192.168.100.20
jorge-glpi@srv-glpi:/opt/portal_tfg$
```

Verificación de que la resolución DNS funciona correctamente desde el servidor GLPI.



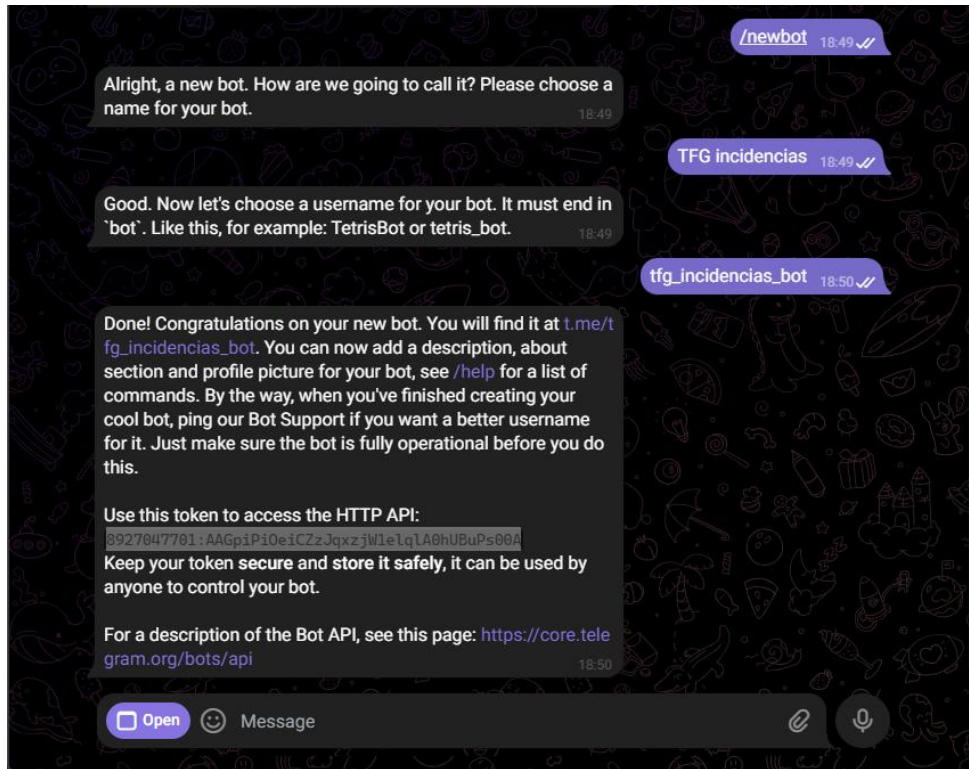
Acceso a GLPI desde el navegador usando el nombre de dominio en lugar de la IP, con HTTPS activo.



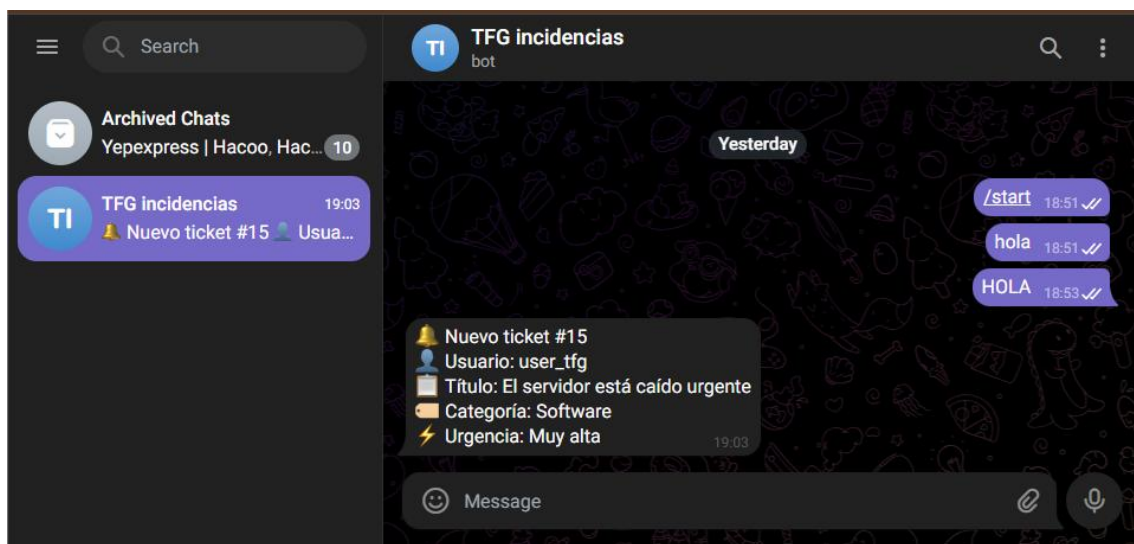
Acceso al portal Flask desde el navegador usando el nombre de dominio con HTTPS activo.

Anexo J — Bot de Telegram y urgencia automática.

En este anexo se documenta la implementación del bot de Telegram y el sistema de clasificación automática de urgencia.



Proceso de creación del bot en Telegram mediante BotFather, mostrando el token de autenticación generado.



Mensaje recibido en Telegram con el número de ticket, usuario, título, categoría y urgencia asignada automáticamente.

Nueva Incidencia Cerrar sesión

Usuario: user_tfg

Título *

El servidor está caído urgente

Descripción *

El servidor está caído urgente

Categoría

Software

Crear incidencia

Formulario del portal tras enviar un ticket mostrando la urgencia que el sistema asignó automáticamente según las palabras clave detectadas.

regla Regla global (+) grupo Buscar ☆

Acciones

ID	TÍTULO	ESTADO	ÚLTIMA ACTUALIZACIÓN	FECHA DE APERTURA	PRIORIDAD	SOLICITANTE - SOLICITANTE	ASIGNADA A - TÉCNICO	CATEGORÍA	TIEMPO PARA RESOLVER
15	El servidor está caído urgente	En curso (asignada)	27-05-2026 17:03	27-05-2026 17:03	Alta			Software	
14	Procesador no pita	Resuelto	22-05-2026 07:29	22-05-2026 07:26	Alta			Hardware	
12	Falta de puertos USB	En curso (asignada)	20-05-2026 18:21	20-05-2026 18:21	Media			Hardware	

15 / 15 filas / página Mostrando 1 a 8 de 8 filas

Verificación en GLPI de que el ticket aparece con la urgencia asignada automáticamente por el sistema de palabras clave.

9. OTROS PUNTOS

a. Aportaciones personales

Lo que más me ha aportado este proyecto no es el conocimiento técnico en sí, sino aprender a resolver problemas de forma ordenada. Hubo momentos en los que llevaba horas atascado con un error y la solución no era la que parecía obvia. El caso más claro fue la integración entre OpenLDAP y GLPI, donde estuve mucho tiempo peleando con el certificado TLS hasta que entendí que el problema estaba en cómo PHP gestionaba la conexión.

Las tres mejoras adicionales que implementé por iniciativa propia, la urgencia automática, el bot de Telegram y el DNS con HTTPS, me han enseñado que un sistema nunca está terminado. Siempre hay algo que se puede mejorar, automatizar o hacer más profesional. La clasificación automática de urgencia me hizo pensar en cómo añadir lógica inteligente a un sistema sin complicarlo en exceso. El bot de Telegram me demostró lo sencillo que es integrar servicios externos cuando tienen una API bien documentada. Y el DNS con HTTPS cerró el proyecto con el nivel de acabado que tendría en una empresa real.

b. Retos profesionales

El mayor reto profesional que veo es escalar esto a una empresa real. Lo que he montado funciona bien como laboratorio, pero con cientos de usuarios necesitarías replicación en el servidor LDAP, balanceo de carga en el servidor web y probablemente una arquitectura basada en contenedores Docker. El bot de Telegram podría evolucionar para permitir al técnico gestionar tickets directamente desde el móvil, sin necesidad de entrar a GLPI. Eso es lo que quiero seguir aprendiendo.

c. Retos personales

Gestionar el tiempo entre el desarrollo técnico, la resolución de problemas y la documentación ha sido el mayor reto personal. El TFG me ha enseñado que documentar mientras desarrollas, no después, es fundamental.

d. Agradecimientos

Quiero agradecer a mi tutor su ayuda y disponibilidad durante todo el proyecto. Sus sugerencias de añadir el backup automático con subida a Dropbox y la asignación automática de tickets por departamento mejoraron mucho el resultado final y le dieron un enfoque más profesional y realista.